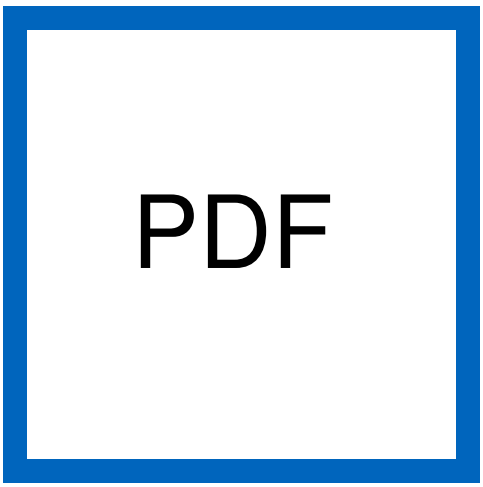
Stereo Camera-Based Environment Perception for Automated Vehicles

Stereokamera-Basierte Umfeldwahrnemung für automatisierte Fahrzeuge



Scientific work for obtaining the academic degree
Master of Science (M.Sc.)
at the Department Mobility Systems Engineering
of the TUM School of Engineering and Design
at the Technical University of Munich

Supervised by	Prof. Dr.-Ing. Markus Lienkamp David Brecht, M.Sc. Chair of Automotive Technology
Submitted by	Adrian Rieker, B.Sc. Schleißheimer Str. 126 80797 München
Submitted on	July 01, 2025

Chair of Automotive Technology
TUM School of Engineering and Design
Technical University of Munich

Project description

Titel der Arbeit

Hier steht ein zum Thema hinführender Text.

In einer konstruktiven (oder theoretischen) Bachelor/-Semester/-Masterarbeit sollen (...)

Folgende Punkte sind durch Herrn/Frau Martin Musterstudent zu bearbeiten:

- Punkt 1
- Punkt 2

Die Ausarbeitung soll die einzelnen Arbeitsschritte in übersichtlicher Form dokumentieren. Der Kandidat/Die Kandidatin verpflichtet sich, die Bachelor/-Semester/-Masterarbeit selbständig durchzuführen und die von ihm verwendeten wissenschaftlichen Hilfsmittel anzugeben.

Die eingereichte Arbeit verbleibt als Prüfungsunterlage im Eigentum des Lehrstuhls.

Announcement date: April 1, 2021

Submission date: October 1, 2021

Prof. Dr.-Ing. Markus Lienkamp

Max Musterbetreuer, M.Sc.

Chair of Automotive Technology
TUM School of Engineering and Design
Technical University of Munich

Geheimhaltungsverpflichtung

Herr: **Rieker, Adrian**

Gegenstand der Geheimhaltungsverpflichtung sind alle mündlichen, schriftlichen und digitalen Informationen und Materialien die der Unterzeichner vom Lehrstuhl oder von Dritten im Rahmen seiner Tätigkeit am Lehrstuhl erhält. Dazu zählen vor allem Daten, Simulationswerkzeuge und Programmcode sowie Informationen zu Projekten, Prototypen und Produkten.

Der Unterzeichner verpflichtet sich, alle derartigen Informationen und Unterlagen, die ihm während seiner Tätigkeit am Lehrstuhl für Fahrzeugtechnik zugänglich werden, strikt vertraulich zu behandeln.

Er verpflichtet sich insbesondere:

- derartige Informationen betriebsintern zum Zwecke der Diskussion nur dann zu verwenden, wenn ein ihm erteilter Auftrag dies erfordert,
- keine derartigen Informationen ohne die vorherige schriftliche Zustimmung des Betreuers an Dritte weiterzuleiten,
- ohne Zustimmung eines Mitarbeiters keine Fotografien, Zeichnungen oder sonstige Darstellungen von Prototypen oder technischen Unterlagen hierzu anzufertigen,
- auf Anforderung des Lehrstuhls für Fahrzeugtechnik oder unaufgefordert spätestens bei seinem Ausscheiden aus dem Lehrstuhl für Fahrzeugtechnik alle Dokumente und Datenträger, die derartige Informationen enthalten, an den Lehrstuhl für Fahrzeugtechnik zurückzugeben.

Eine besondere Sorgfalt gilt im Umgang mit digitalen Daten:

- Für den Dateiaustausch dürfen keine Dienste verwendet werden, bei denen die Daten über einen Server im Ausland geleitet oder gespeichert werden (Es dürfen nur Dienste des LRZ genutzt werden (Lehrstuhllaufwerke, Sync&Share, GigaMove).
- Vertrauliche Informationen dürfen nur in verschlüsselter Form per E-Mail versendet werden.
- Nachrichten des geschäftlichen E-Mail Kontos, die vertrauliche Informationen enthalten, dürfen nicht an einen externen E-Mail Anbieter weitergeleitet werden.
- Die Kommunikation sollte nach Möglichkeit über die (my)TUM-Mailadresse erfolgen.

Die Verpflichtung zur Geheimhaltung endet nicht mit dem Ausscheiden aus dem Lehrstuhl für Fahrzeugtechnik, sondern bleibt 5 Jahre nach dem Zeitpunkt des Ausscheidens in vollem Umfang bestehen. Die eingereichte schriftliche Ausarbeitung darf der Unterzeichner nach Bekanntgabe der Note frei veröffentlichen.

Der Unterzeichner willigt ein, dass die Inhalte seiner Studienarbeit in darauf aufbauenden Studienarbeiten und Dissertationen mit der nötigen Kennzeichnung verwendet werden dürfen.

Datum: 1. Juli 2025

Unterschrift: _____

Chair of Automotive Technology
TUM School of Engineering and Design
Technical University of Munich

Erklärung

Ich versichere hiermit, dass ich die von mir eingereichte Abschlussarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Garching, den 1. Juli 2025

Adrian Rieker, B.Sc.

Chair of Automotive Technology
TUM School of Engineering and Design
Technical University of Munich

Declaration of Consent, Open Source

Hereby I, Rieker, Adrian, born on August 14, 2000, make the software I developed during my Masterthesis Thesis available to the Institute of Automotive Technology under the terms of the license below.

Garching, July 1, 2025

Adrian Rieker, B.Sc.

Contents

Formula Symbols

Formula Symbols	Unit	Description
f_x	px	Focal length of the camera in the x direction
f_y	px	Focal length of the camera in the y direction
c_x	px	x coordinate of the principal point
c_y	px	y coordinate of the principal point
$\mathbf{x}_k$		State vector at time k (3D position and velocity)
$\hat{\mathbf{x}}_k$		Predicted state vector at time k
$\mathbf{P}_k$		State covariance matrix
$\hat{\mathbf{P}}_k$		Predicted state covariance matrix
$\mathbf{A}$		State transition matrix
$\mathbf{B}$		Control input matrix
$\mathbf{u}_k$		Control input vector (e.g. ego-motion)
$\boldsymbol{\omega}_k$		Process noise vector
$\mathbf{Q}$		Covariance matrix of process noise
$\mathbf{z}_k$		Measurement vector (e.g. pixel + depth)
$\hat{\mathbf{z}}_k$		Predicted measurement vector
$\mathbf{H}$		Measurement matrix (linear case)
$\mathbf{H}_k$		Jacobian of the nonlinear measurement function
$\mathbf{h}(\cdot)$		Nonlinear measurement function
$\mathbf{R}$		Measurement noise covariance matrix
$\hat{\mathbf{T}}_k$		Predicted measurement covariance
$\mathbf{s}_k$		Innovation vector (residual)
$\mathbf{S}_k$		Innovation covariance matrix
$\mathbf{K}_k$		Kalman gain
$\mathbf{I}$		Identity matrix
u_k	px	Horizontal pixel coordinate

v_k	px	Vertical pixel coordinate
d_k	m	Depth value
Δt	s	Time difference between frames

Todo list

1 Introduction

Autonomous vehicles are rapidly becoming a reality, offering the promise of safer and more efficient transportation. However, their adoption still faces significant challenges, particularly in the area of perception. These systems are responsible for detecting and interpreting objects in the vehicle's surroundings, a task that becomes increasingly difficult in dynamic, unstructured, or unforeseen environments. In such cases, perception failures may lead to disengagements, where the autonomous system relinquishes control and requires remote human assistance via teleoperation.

In these critical moments, the reliability of the perception system is most important. It must provide robust, real-time awareness of the environment to either enable safe fallback strategies or support the teleoperator in navigating through complex scenes.

Most existing perception systems, responsible for detecting and tracking objects in the vehicle's surroundings, rely heavily on learning-based solutions. While these methods have demonstrated impressive performance in structured and controlled environments, they often struggle to generalize in novel and untrained scenarios. This limitation is particularly problematic in safety-critical situations, where the vehicle must make quick decisions based on the perceived environment.

This project goal is the development of a deterministic, stereo camera based perception system for autonomous vehicles, capable of providing a reliable and robust understanding of the surroundings, specially in highly dynamic and unforeseen scenarios. The system will take as input two images from a stereo camera mounted on the vehicle and output a list of detected objects in the scene, along with their criticality scores, forming a situational awareness module to assess potential hazards. Additionaly, this deterministic approach could run in parallel with existing learning-based perception systems, providing a an additional layer of redundancy and security in critical situations where the learning-based methods may fail.

To achieve accurate 6D motion estimation (position + velocity), the system will implement a deterministic algorithm inspired by the Daimler 6D Vision system. This approach uses Kalman Filters to estimate a 6D state vector for selected feature points in the scene, which will then be clustered to identify moving objects. Unlike learning-based methods, this approach does not require extensive training data and can generalize well in new unseen scenarios.

The perception framework is implemented in **C++** using the **Robot Operating System 2 (ROS2)** to ensure real-time capability, modularity, and easy deployment. ROS2 also enables seamless integration with the software stack of the **EDGAR research vehicle**, where the system will be deployed and tested. The modular design allows each component to run as an individual node, facilitating parallel processing and containerized using **Docker**.

The perception system is structured in two main pipelines:

- **The perception pipeline**, which computes dense optical flow, estimates ego-motion via stereo visual odometry, and derives depth maps from the stereo camera. These are fused in a **6D Kalman Filter** to estimate the position and velocity of selected points in the scene. The resulting motion field is then clustered into discrete objects using the **DBSCAN** algorithm.

- **The criticality assessment pipeline**, which takes the list of moving objects and the vehicle's planned trajectory to compute runtime safety metrics, particularly TTC. This module is crucial for evaluating risks and supporting decision-making in safety-critical situations.

The system makes extensive use of GPU acceleration via **CUDA**, particularly in modules such as optical flow estimation and depth computation. This enables the system to meet the performance requirements of real-time processing on the EDGAR platform.

Compared to previous works, such as the Daimler 6D Vision system, this approach integrates the deterministic motion estimation pipeline into a broader perception architecture tailored for real-world deployment in autonomous vehicles. Furthermore, by combining stereo vision, visual odometry, and deterministic filtering, the system is able to operate without any pretraining or external semantic data, increasing transparency and robustness.

To validate the system, a two-stage evaluation will be conducted. First, each module will be tested individually using public datasets such as KITTI, which provides ground truth for depth, optical flow, and odometry. Then, the integrated pipeline will be deployed in the EDGAR vehicle and evaluated in urban and semi-structured environments. Its outputs will be compared to the perception stack currently in use on the vehicle, which is based on deep learning models.

In summary, this work contributes a real-time capable, modular, and interpretable perception system based on stereo vision and deterministic algorithms. By improving robustness in edge-case scenarios and supporting redundancy in perception, the system aims to increase safety and reliability in the operation of autonomous vehicles.

2 State of the Art

Autonomous and automated vehicles must have a comprehesive and accurate understanding of their surroundings at all times to ensure safe and reliable operation. This process, known as environment perception, is responsible for detecting, identifying, and tracking objects such as other vehicles, cyclists, pedestrians or other road elements. The quality of the measurement of the state of this objects directly influence the vehicle's ability to make safe and informed driving decisions.

Failures in the perception can lead to hazarous situations, such as collisions or unexpected navigation decisions. This is particularly critical in high-speed scenarios or complex urban environments, where is a high density of high dynamic objects which make more complex the operation in real time.

A robust perception system must provide the following capabilities:

1. **Object detection:** Identify and locate objects in the vehicle's surroundings, such as vehicles, pedestrians, cyclists, or other road elements.
2. **Motion estimation:** Determining the position and velocity of moving objects
3. **Scene understanding:** Classifiying the detected objects and assessing their criticality for the vehicle's navigation
4. **Reliability in edge cases:** Ensuring correct operation in complex and unforeseen scenarios, such as occlusions, adverse weather conditions, or sensor failures.
5. **Real-time operation:** Providing timely and accurate information to the vehicle's control system to make informed driving decisions.

While the perception concept includes different aspects, this work will focus specifically on the detection of moving object in the vehicle's surroundings and the estimation of their 6D motion state (position and velocity) using a deterministic approach. In the following sections, we will provide an overview of the state-of-the-art of the different sensors and algorithms used in the perception systems of autonomous vehicles, focusing on the deterministic image based approaches.

2.1 Perception Systems for Autonomous Vehicles

The perception systems of autonomous vehicles rely on a combination of different sensors to provide a comprehensive understanding of the ego vehicle's surroundings. In this section we will provide an overview of the most common sensor types used in autonomous vehicles and their advantages and limitations.

LiDAR-Based Perception

LiDAR (Light Detection and Ranging) sensors are increasingly used in the automotive industry for obstacle detection, 3D mapping and lane detection [1]. More and more companies are adopting this type of sensor due

to its advantage in rapidly and accurately estimating 3D distances. Examples include BMW [2], Merceces-Benz [3], and Waymo [4].

Working principle LiDAR sensors use the concept of time of flight to measure the distance to objects in the vehicle's surroundings. The sensor emits a laser beam and measures the time it takes for the light to reflect back to the sensor. By calculating the time difference, the sensor can determine the distance to the object based on the speed of light [5].

Advantages and Limitations LiDAR sensors have the advantage of providing an accurate 3D representation of the vehicle's surroundings. This makes them particularly useful for object detection and localization tasks. However, LiDAR sensors are sensitive to adverse weather conditions, such as rain, snow or fog since the laser beams can be scattered by the particles in the air. Additionally, LiDAR sensors are more expensive than other sensor types, such as cameras or radars, which can limit their adoption in mass-market vehicles, although their prices are steadily decreasing [1].

Radar-Based Perception

Radar (Radio Detection and Ranging) use radio waves to detect objects in the vehicle's surroundings. They are commonly used in the automotive industry for object avoidance.

Working principle Similar to LiDAR sensors, radar sensors emit radio waves and measure the time it takes for the waves to reflect back to the sensor. By calculating the time difference, the sensor can determine the distance to the object based on the speed of light. Radar sensors can also estimate the object's velocity based on the Doppler effect [6].

Advantages and Limitations Radar sensors are less sensitive to adverse weather conditions than LiDAR sensors, making them more reliable in rain, snow, or fog. [1, 7]. However, radar sensors have a lower resolution than LiDAR sensors, which can limit their ability to detect small objects or provide detailed information about the object's shape.

Image-Based Perception

Image based perception systems use cameras to capture images of the vehicle's surroundings and are the principal sensor in some companies like Tesla [8]. These sensors can be used as single cameras for object detection and classification or in pairs to provide stereo vision, which allows for depth estimation and 3D reconstruction of the scene. These type of sensors can also be used for optical flow computation and the ego motion estimation through visual odometry

Monocular vision Monocular vision systems use a single camera to capture images of the vehicle's surroundings. These systems are commonly used for object detection and classification tasks, such as detecting pedestrians, cyclists, or other vehicles [1]. Monocular vision systems are less expensive than stereo vision systems and can be easily integrated into existing vehicles. However, they have limitations in depth perception and 3D reconstruction, which can make it challenging to estimate the distance to objects accurately.

Stereo vision Stereo vision systems use two cameras to capture images of the vehicle's surroundings and use the pinhole camera model [9] to estimate the depth of objects in the scene. By comparing the images from the two cameras, the system can calculate the disparity between corresponding points in the images and use this information to estimate the distance to the objects.

Advantages and Limitations Image-based perception systems have the advantage of providing a rich and detailed information about the environment. Cameras can capture color information, texture, and shape of objects, which can be useful for object detection and classification tasks. In addition, cameras are less expensive than LiDAR sensors and can be easily integrated into existing vehicles. However, image-based perception systems are sensitive to adverse weather conditions, such as rain, snow, or fog, which can affect the quality of the images and limit the system's performance. The algorithms used for object detection and tracking in image-based perception systems can also be computationally intensive, which can affect the system's real-time performance.

In the following table a summary of the advantages and limitations of the different sensor types used in autonomous vehicles is provided:

Table 2.1: Summary of sensor types used in autonomous vehicles

Sensor type	Advantages	Limitations
LiDAR	Accurate 3D representation	Expensive and sensitive to adverse weather conditions
Radar	Works in bad weather, detects velocity	Low spatial resolution
Cameras	Rich scene details, color information, Cheaper than the other considered sensors	Depth perception is limited, affected by lighting, expensive computation cost

While sensors provide raw data about the environment, perception algorithms are responsible for interpreting this information to detect and track moving objects. These algorithms can be broadly classified into learning-based and deterministic approaches, as discussed in the following sections.

2.2 Algorithms for moving object detection

The perception algorithms can also be classified into two main categories: learning-based and deterministic approaches. Learning-based approaches use machine learning techniques, such as deep learning, and are data driven. In the other side, deterministic approaches use mathematical models to estimate the state of the objects in the scene. In the following sections we will provide an overview of the most common algorithms used for moving object detection in autonomous vehicles.

Learning-Based Approaches

Learning-based approaches have gained popularity in recent years. These methods use convolutional neural networks (CNNs), recurrent neural networks (RNNs), and transformers to process the data coming from cameras, LiDARs, and radar systems. In the work done by Wen and Jo [10], they provide a comprehensive review of the different learning-based approaches used in autonomous vehicles, which can be summarized in the following figure:

Some limitations of learning-based approaches are the need for extensive training data, which can be expensive and time-consuming to collect. These methods can also struggle to generalize in novel and untrained scenarios, which can be problematic in safety-critical situations where the vehicle must make quick decisions based on the perceived environment.

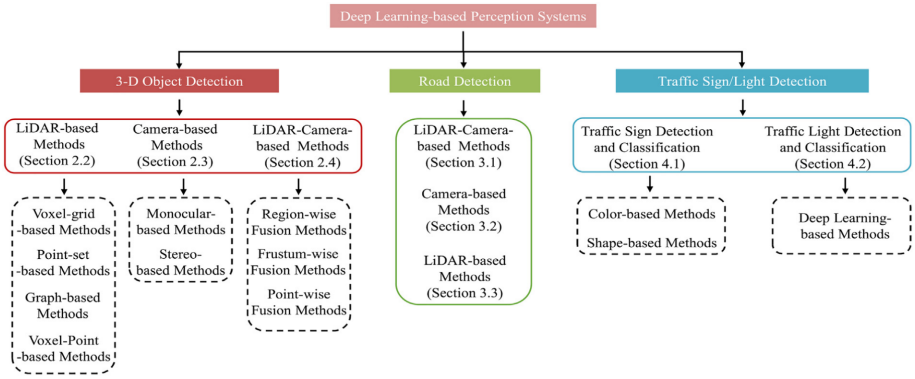


Figure 2.1: Learning-based perception methods. Source: [10]

Deterministic Approaches

Deterministic approaches use mathematical models to estimate the state of the objects in the scene.

Optical flow Optical flow is a technique used to estimate the motion of objects in the scene by tracking the movement of pixels between consecutive frames. This method is based on the assumption that the intensity of a pixel remains constant over time, which allows the system to track the movement of objects based on the changes in pixel intensity [11]. Some of the most common optical flow algorithms are the Lucas-Kanade method [12] and the Horn-Schunck method [**flow_schunck**].

6D Vision 6D vision is a deterministic perception framework that combines stereo vision and motion tracking to estimate the 3D position and velocity of objects in the scene. Originally introduced by Franke et all [13] this method provides a direct motion estimation without requiring object segmentation. The key principles of 6D Vision are:

1. **Stereo disparity estimation:** Calculate the disparity between corresponding points in the stereo images to estimate the depth of objects in the scene.
2. **Optical flow tracking:** Track the movement of feature points in the scene between consecutive frames.
3. **Kalman filtering:** Use Kalman filters to estimate the 6D state vector (position and velocity) of the feature points in the scene.

2.3 State of the Art in the FTM

The Institute of Automotive Technology (FTM) at the Technical University of Munich has conducted previous research in the field of environment perception for autonomous and automated vehicles. This section provides an overview of previous work carried out at the FTM, particularly regarding the EDGAR research vehicle and the development of deterministic perception systems based on 6D vision.

2.3.1 EDGAR Research Vehicle

The EDGAR (Excellent Driving Garching) research vehicle is a state-of-the-art autonomous driving platform developed at FTM [14]. It serves as a testbench for autonomous driving research, including perception, planning, control, and sensor fusion. The vehicle is equipped with a multi-sensor setup, comprising:

1. **Stereo and monocular cameras:** For image-based perception and object detection
2. **LiDAR sensors:** For 3D mapping and obstacle detection
3. **Radar sensors:** For object avoidance and velocity estimation
4. **GPS/IMU:** For localization and motion tracking

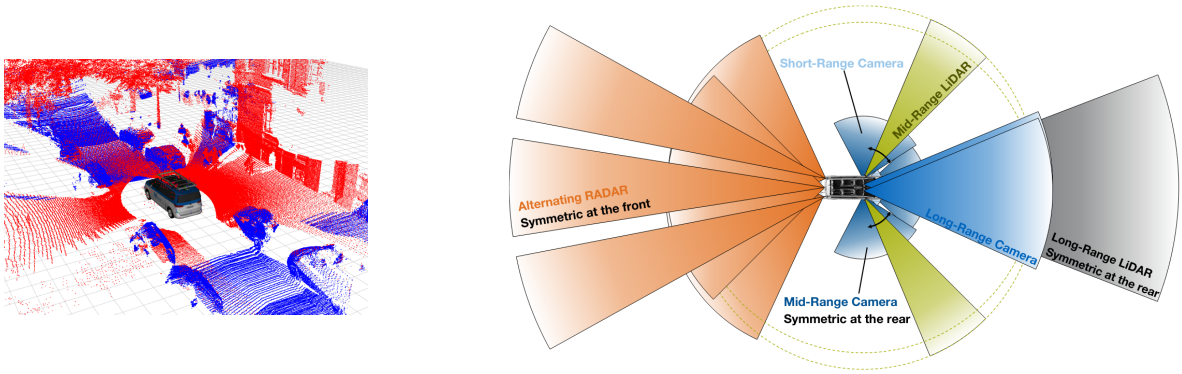


Figure 2.2: EDGAR Vehicle Sensors

2.3.2 Previous Work in the FTM

This project builds upon previous work conducted at the FTM in the field of environment perception for automated vehicles with 6D vision. Some of the key contributions include:

1. **Objektdetektion und Bewegungsprädiktion mittels eines Stereokamerasystems beim teleoperierten Fahren** [15]: Bauer in his master's thesis developed a deterministic system for object detection based on the 6D vision of Daimler [13]. The system was implemented in C++ using the ADTF framework Automotive Data and Time-Triggered Framework and was capable of detecting and tracking objects in the vehicle's surroundings with some real time limitations.
2. **Weiterentwicklung und echtzeitfähige Umsetzung eines Stereo-Vision-basierten Bewegungsprädiktionssystems für die aktive Sicherheit von teleoperierten Fahrzeugen** Some time later in [16] Rotar continued Bauers work by implementing the system in GPU using CUDA to speed up the computationally intensive tasks.
3. **Validation of an Obstacle Detection Approach for Teleoperation of Automated Vehicles:** Finally Pastor [17] started porting the system to ROS1 to facilitate the integration with other modules in the EDGAR vehicle. The system was implemented without CUDA GPU acceleration and was not deployed in the EDGAR vehicle. The system also has some problem with the object detection.

Building upon these previous contributions, this project aims to further improve deterministic perception by addressing key limitations found in prior implementations. In particular, this work will focus on enhancing

computational efficiency, real-time feasibility, and system deployment within ROS2. These improvements will be detailed in the next section on objectives.

2.4 Stereo camera technology

Short review on stereo camera technology.

2.5 Methods for software containerization

Short review on methods for software containerization.

2.6 Requirements for the Thesis

Short review on the requirements for the thesis.

3 Methods and Implementation

This chapter outlines the methodological and technical approaches that form the foundations of this project. It begins with a high level architecture of the system, providing insight into how the different modules interact to enable real-time perception and criticality assessment for automated vehicles.

Following this, each component of the systme is examined in detail, from the underlying algorithms to the some of the implementation details. Emphasis is placed on the design choices made to ensure robustness, real-time performance, and modularity.

By decomposing the system into well-defined modules, this architecture enables both scalability and flexibility for experimentation, while maintaining consistent performance in real-world conditions. It also facilitates further development and seamless integration with other components of the EDGAR vehicle's software stack.

3.1 System Architecture

The system is conceptually divided into two main pipelines: the perception pipeline and the criticality pipeline. The **perception pipeline** processes raw sensor data captured by the stereo camera to build a dynamic representation of the environment and estimating the position and speed vectors of the moving objects. The output of this pipeline is a list of object in the scene that is later used by the **criticality pipeline** to compute the criticality score of the detected objects.

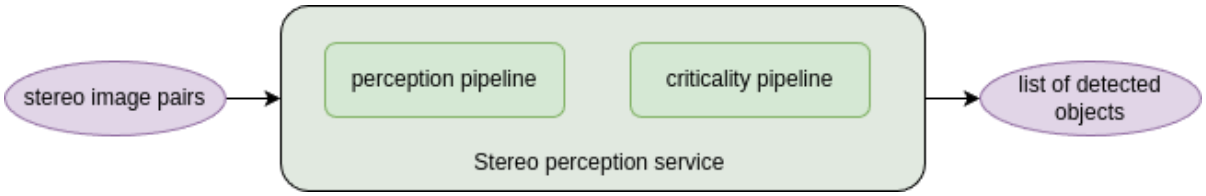


Figure 3.1: High level overall system

3.1.1 Perception pipeline

The perception pipeline will be responsible for detecting moving objects and estimating their position and current velocity. The pipeline consists of several submodules as illustrated in figure 3.2.

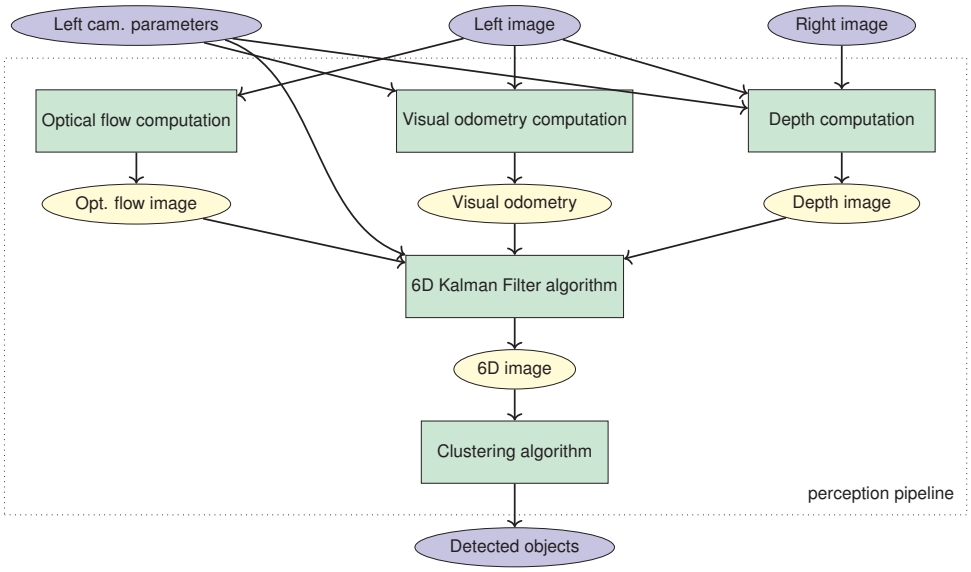


Figure 3.2: High level perception pipeline

Search source of sgm algorithm

As shown in the figure, the perception pipeline comprises the following core modules:

1. **Optical flow computation:** Calculates a dense optical flow field from the left image sequence using the Farneback algorithm.
2. **Visual odometry:** Estimates the ego-motion of the vehicle using stereo image sequences
3. **Depth computation:** Computes a dense depth image from the stereo image pair using a semi-global matching (SGM) algorithm.
4. **Kalman filter:** Central module of the perception pipeline. It uses depth and optical flow data to estimate the 6D state (position and velocity) of a sparse set of points over time.
5. **Object detection:** Applies a clustering algorithm (DBSCAN) to group similar 6D points into detected objects.

3.1.2 Criticallity pipeline

The criticality pipeline (Figure 3.3) assesses the risk posed by each detected object. Based on the output of the perception pipeline and the current trajectory of the ego-vehicle, it computes criticality metrics such as the Time-to-Collision (TTC) [18]. These metrics serve as indicators of potential hazards in the environment.

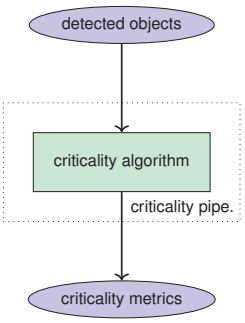


Figure 3.3: High level criticality pipeline

3.1.3 Repository structure

The project is implemented as a monorepo, consolidating all system components into a single, unified repository to simplify development, integration, and deployment.

The repository is structured into three main directories:

- **src:** Contains all ROS 2 source code. It is further divided into:
  - **launch_packages**: Includes ROS 2 launch files used to start multiple components together.
  - **perception_pipeline**: Contains the core modules of the perception pipeline, including **camera_info_publisher**, **stereo_computation**, **optical_flow_computation**, **kalman_filter**, and **object_detector**.
  - **criticality_pipeline**: Contains the **ttc_calculator** package responsible for computing Time-to-Collision and other criticality metrics.
  - **utils**: Provides auxiliary packages and shared message definitions in **stereo_perception_msgs**.
- **scripts:** Includes helper bash scripts and Dockerfiles used to build, run, and deploy the system.
- **config/dds:** Contains configuration files related to middleware settings such as DDS profiles.

An overview of this structure is shown in Figure 3.4.

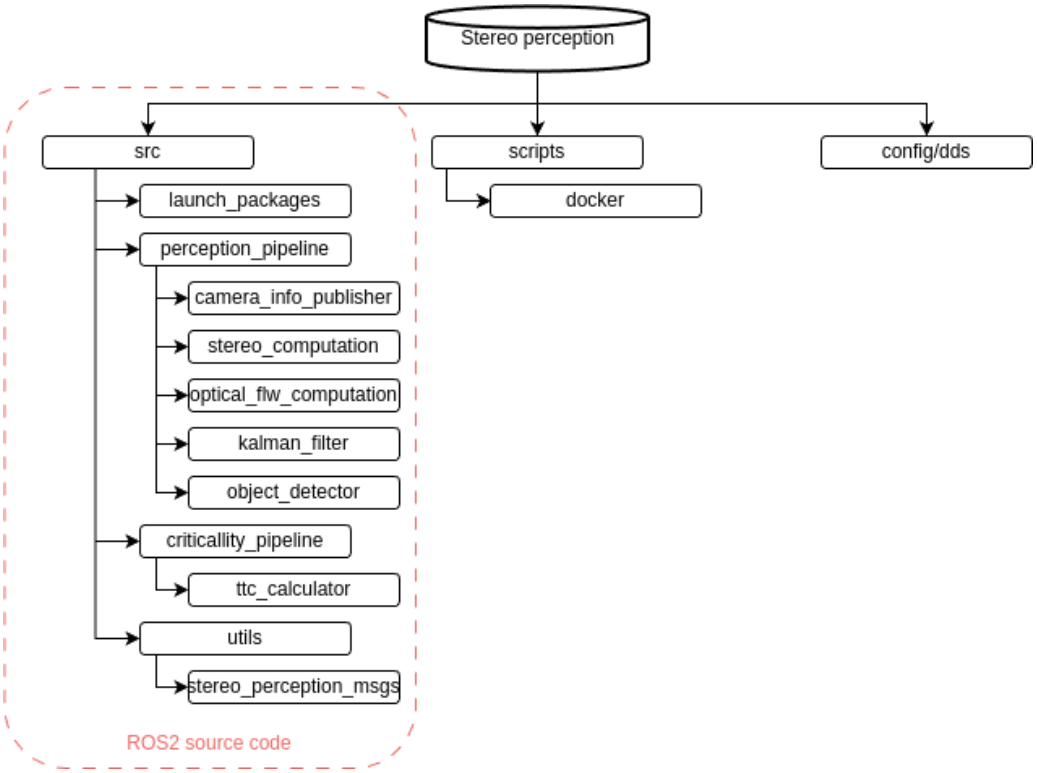


Figure 3.4: Overview of the repository structure

3.1.4 Containerization

Docker is a platform for developing, shipping, and running applications in lightweight, isolated environments called containers. Containers package the application code together with its dependencies and runtime, ensuring consistent behavior across different systems and deployment contexts.

To ensure reproducibility, portability, and streamlined deployment, the entire system is encapsulated within a single Docker container. This design choice is driven by both functional and performance considerations.

Firstly, the system is architected as a unified perception service that processes synchronized stereo camera inputs to produce a list of detected objects along with their associated criticality scores. Given the tight coupling and interdependencies among the modules, segregating components into multiple containers would introduce unnecessary inter-process communication overhead.

Secondly, employing a single container mitigates latency introduced by serialization and network layers between containers. This is particularly crucial in real-time systems, where maintaining low processing latency is imperative. Running all modules in the same memory space allows for efficient data exchange and eliminates potential bottlenecks associated with inter-container messaging.

This containerization strategy also simplifies integration with the research platform, facilitating easier deployment, logging, and testing within a controlled and reproducible environment.

Explain how it was built: OpenCV, CUDA, ROS2, etc.

3.2 Perception pipeline: Stereo image computation

The stereo image computation module is responsible for generating a dense depth map from a pair of rectified stereo images and its calibration parameters. This component is particularly relevant when working with datasets or sensors that provide raw stereo image pairs but no precomputed depth information. Although this module is not used in the final deployment of the system on the EDGAR research vehicle, since its stereo camera setup provides depth images directly—it plays a crucial role in training, testing, and benchmarking the system using external datasets. It can also be used in the future to integrate other stereo camera sets that do not provide depth images.

3.2.1 Mathematical Background: Pinhole Camera Model and Disparity

In stereo vision, depth estimation relies on the triangulation of corresponding points in a pair of images captured from slightly different viewpoints. The pinhole camera model (figure: 3.5) is commonly used to describe the relationship between 3D world coordinates and 2D image coordinates. In this model, a 3D point $\mathbf{P} = (X, Y, Z)$ is projected onto the image plane as a 2D point $\mathbf{p} = (x, y)$ using the camera's intrinsic parameters ([9]). These parameters include the focal length f and the principal point (c_x, c_y), which define the camera's optical center and field of view.

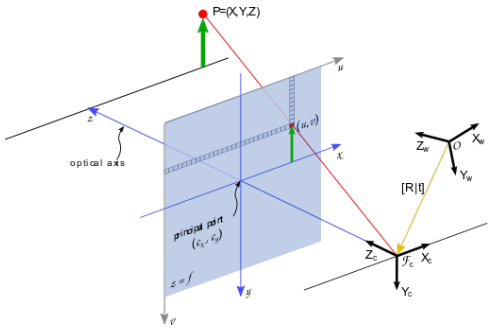


Figure 3.5: Pinhole camera model. Source: Nvidia

The relationship between the 3D point and its 2D projection is given by:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \tag{3.1}$$

where f_x and f_y are the focal lengths in pixels, and (c_x, c_y) is the principal point.

In a typical stereo setup, the two cameras are displaced horizontally along the x-axis, with their optical axes kept parallel. This physical separation between the cameras is known as the **baseline** B, as seen in the figure 3.6 left. The baseline is a key parameter in stereo vision, as it directly influences the triangulation accuracy: a larger baseline improves depth resolution, especially for distant objects, but also increases the difficulty of finding correct correspondences due to greater viewpoint differences.

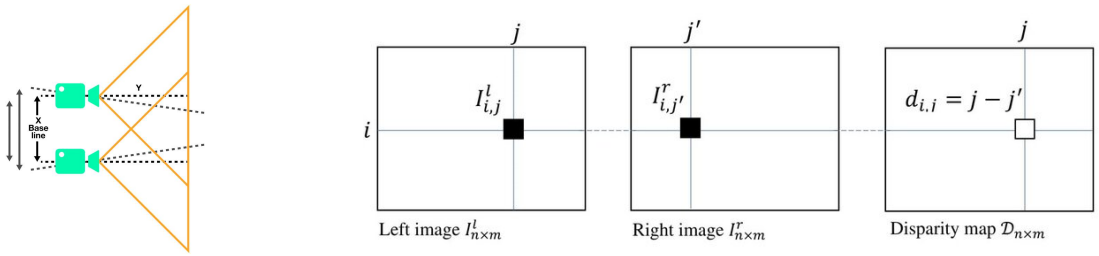


Figure 3.6: Stereo camera setup (left) and disparity concept (right). Source: Framos, [19].

Thanks to this horizontal arrangement, the correspondence search simplifies to a one-dimensional problem along the image rows. After rectification, corresponding points between the two images lie along the same horizontal line (epipolar line), and the **disparity** d (figure 3.6 right) between corresponding points is defined as:

$$d = x_{\text{left}} - x_{\text{right}} \tag{3.2}$$

Disparity is inversely proportional to the depth of the observed point. The relation between disparity and depth Z is given by:

$$Z = \frac{f \cdot B}{d} \tag{3.3}$$

3.2.2 Stereo Matching Algorithms

Stereo matching algorithms are responsible for computing the disparity map between a pair of rectified stereo images. Given that disparity directly relates to depth, these algorithms are a critical component of the stereo vision pipeline. Their main task is to find, for each pixel in the left image, the corresponding pixel in the right image along the same horizontal line.

Several approaches exist to solve this correspondence problem, each with different trade-offs between computational complexity, robustness, and accuracy. Some of the most common implemented stereo matching algorithms include:

- **Block Matching (BM):** A local method that compares fixed-size blocks between the left and right images. While computationally efficient, it often struggles with textureless regions and depth discontinuities ([20]).
- **Semi-Global Matching (SGM):** An algorithm that balances accuracy and computational efficiency by aggregating matching costs along multiple paths through the image ([21]).

Comparative studies ([22]) have shown that SGM offers a good trade-off between accuracy and performance, making it suitable for real-time applications where both speed and quality are essential.

3.2.3 Semi-Global Matching (SGM)

Semi-Global Matching (SGM) is a stereo matching algorithm that offers a compromise between the accuracy of global optimization methods and the computational efficiency of local methods. It was first introduced by Hirschmüller [21] and has become widely implemented in real-time applications.

The underlying principle in SGM is to minimize a global energy function that imposes continuity in the disparity map, having strong depth discontinuities along object boundaries. The energy function $E(D)$ for a disparity map D is defined as:

$$E(D) = \sum_{\mathbf{p}} \left(C(\mathbf{p}, D_{\mathbf{p}}) + \sum_{\mathbf{q} \in \mathcal{N}_{\mathbf{p}}} P_1 \cdot \delta\left(|D_{\mathbf{p}} - D_{\mathbf{q}}| = 1\right) + P_2 \cdot \delta\left(|D_{\mathbf{p}} - D_{\mathbf{q}}| > 1\right) \right) \tag{3.4}$$

where:

- $C(\mathbf{p}, D_{\mathbf{p}})$ is the matching cost at pixel $\mathbf{p}$ for disparity $D_{\mathbf{p}}$.
- $\mathcal{N}_{\mathbf{p}}$ denotes the set of neighboring pixels.
- P_1 and P_2 are penalty terms for disparity changes of 1 and greater than 1, respectively.
- δ is the Kronecker delta function.

The algorithm operates in two main steps:

1. Cost computation: For each possible disparity, a matching cost calculation happens between the left picture plus the right picture at each pixel. This calculation uses methods like the Sum of Absolute Differences or measurements that use census transform.

2. Cost aggregation: SGM does not do a full 2D global optimization. That process demands a lot of computation, so it is not practical for systems that need to run in real time. SGM comes close to global reasoning when it combines costs along several 1D paths. Costs are put together along either 4 or 8 directions from each pixel. A dynamic programming method finds the lowest local cost in each direction. It also adds penalties for significant changes in disparity.

Once the cost values are added together from all directions, the disparity picked for each pixel is the one that has the lowest combined cost.

This balance between accuracy and computational efficiency makes SGM highly attractive for embedded and real-time applications.

3.2.4 ROS2 Implementation

The SGM algorithm has been integrated into the pipeline with a dedicated ROS2 package called `stereo_computation`.It contains a node that subscribes to three topics: the rectified left and right camera images, and the camera info topic of the left camera. After receiving synchronized input, the node computes both the disparity map and the depth map, which are then published on two separate ROS2 topics for downstream processing.

The stereo matching algorithm was implemented using the OpenCV library. In particular, the `cv::cuda::StereoSGM` class was chosen to compute the disparity map. This class offers a CUDA-accelerated version of the Semi-Global Matching algorithm, enabling the computations to be performed directly on the GPU. Taking advantage of GPU acceleration is crucial to meet the real-time performance requirements of the perception pipeline.

The OpenCV implementation of SGM is highly optimized and designed to exploit the parallel architecture of modern GPUs, significantly reducing the runtime compared to CPU-based methods. After computing the disparity map, the depth map is generated by applying the classical pinhole camera equation:

$$Z = \frac{f \cdot B}{d} \tag{3.5}$$

where Z is the depth value, f is the focal length of the camera, B is the baseline distance between the two cameras, and d is the disparity.

This node architecture ensures modularity within the perception pipeline and allows easy integration with other modules, such as the Kalman filtering and object detection components. Furthermore, by publishing both the disparity and depth images, the system maintains flexibility for future extensions or algorithm improvements that may require direct access to raw disparity data.

To make the tuning and implementation easier, the following ROS2 parameters were added to the node:

ROS2 parameter	Definition
in_left_image_topic	Topic name for the left input grayscale image
in_right_image_topic	Topic name for the right input grayscale image
in_camera_info_topic	Topic name for the camera calibration information
out_depth_image_topic	Output topic for the computed depth image
out_disparity_image_topic	Output topic for the computed disparity image
min_disparity	Minimum disparity to consider for the disparity calculation
num_disparities	Number of disparities (disparity range) to compute
P1	Control parameter for the SGM algorithm (penalty for small disparity changes)
P2	Control parameter for the SGM algorithm (penalty for small disparity changes)
uniqueness_ratio	Threshold for uniqueness of matching points (higher means stricter matching)
speckle_window_size	Size of the window used for speckle filtering
speckle_range	Maximum disparity range for speckle filtering
pre_filter_cap	Pre-filtering parameter for the image to improve disparity computation
use_sim_time	Flag to use simulation time instead of system time

and the node can be added to a launch file as seen in the following example:

Code 3.1: Stereo computation launch

```
launch_ros.actions.Node(
            package='stereo_computation',
            executable='stereo_computation_node',
            output='screen',
            parameters=[
              {"in_left_image_topic": input_gray_left_topic},
              {"in_right_image_topic": input_gray_right_topic},
              {"in_camera_info_topic": input_camera_info_topic
 },
              {"out_depth_image_topic": "perception_pipeline/
 depth_image"},
              {"out_disparity_image_topic": "
 perception_pipeline/disparity"},
              {"min_disparity": 0},
              {"num_disparities": 256},
              {"P1": 30},
              {"P2": 200},
              {"uniqueness_ratio": 15},
              {"speckle_window_size": 50},
```

```
17                 {"speckle_range": 2},
18                 {"pre_filter_cap": 31},
19                 {"use_sim_time": use_sim_time}
20                 ],
21             ),
```

3.3 Perception pipeline: Optical flow computation

This section describes the optical flow computation module within the perception pipeline. This module is responsible for estimating the relative motion of the pixels of the image between two consecutive frames.

3.3.1 Optical flow concept

The concept of Optical Flow, first introduced by James Gibson in the 1950s ([23]) refers to the displacements of intensity patterns. In our case it represents the relative motion of the pixels between two consequtive frames of a video stream. The output is a 2D vector field where each vector describes the position of a pixel or a small region in the current frame relative to its position in the previous frame.

This module is crucial for estimating the information of how the pixels in the image are moving, which is one of the essential inputs to the Kalman Filter stage.

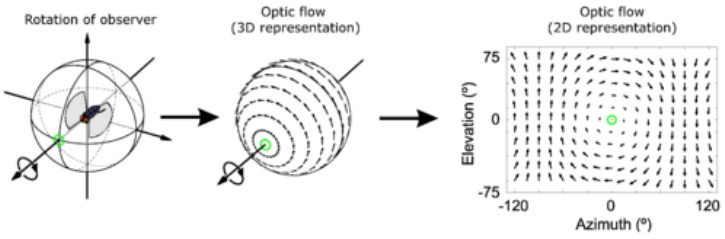


Figure 3.7: Optical flow

Optical flow estimation relies on the assumotion that the brightness of a pixel remains constant over time [24, 25], which allows the system to track the movement of objects based on the changes in pixel intensity. Mathematically, this assumption leads to the optical flow constraint equation:

$$I_x u + I_y v + I_t = 0 \tag{3.6}$$

Where:

- I_x and I_y are the spatial gradients of the image intensity in the x and y directions.
- I_t is the temporal gradient of the image intensity (the difference between two consecutive frames).
- u and v are the optical flow components in the x and y directions.

Since there is one equation and two unknowns, the optical flow constraint equation is underdetermined. To solve this problem, additional constraints or assumptions are needed, leading to different optical flow algorithms.

3.3.2 Optical flow algorithms

Optical flow methods can be classified into two main categories: sparse and dense optical flow.

Sparse optical flow methods focus on estimating motion vectors at a discrete set of key points in the image. These points are typically features such as corners or edges that can be tracked reliably across frames. In contrast, **dense optical flow** methods aim to estimate a motion vector for every pixel in the image, providing a complete 2D displacement field.

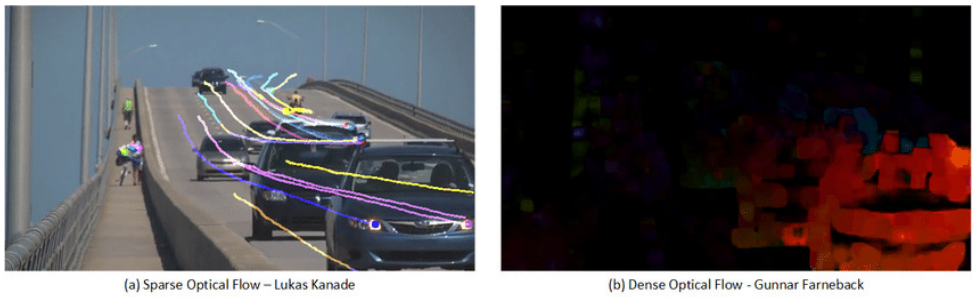


Figure 3.8: Sparse vs dense optical flow. Source: [26]

Recently, deep learning-based methods have gained popularity and demonstrated impressive performance for optical flow estimation. Models such as FlowNet [27] and RAFT [**opt_flw_raft**] leverage convolutional neural networks (CNNs) to learn spatial and temporal features from large datasets, enabling them to predict optical flow fields with high accuracy. However, these methods often require significant computational resources and may suffer from poor generalization to unseen environments.

For these reasons, and due to the necessity of robustness and real-time performance, this work focuses on traditional deterministic optical flow algorithms. These classical methods also offer the advantage of being widely implemented and optimized in existing libraries, which significantly accelerates development and integration time.

Some of the most common optical flow algorithms include [11, 28]:

- **Lucas-Kanade method [12]:**

  Introduces in 1981 by Lucas and Kanade, this os a sparse optical flow method based on the assumption of constant optical flow within a small local neighborhood. It solves the optical flow constraint equation using a least-squares approach. This method is simple and computationally efficient, but it is sensitive to large motions and requires good texture in the tracked regions.

- **Horn-Schunck method [29]:**

  Proposed by Horn and Schunck, this method computed dense optical flow by introducing a global smoothness constraint. It formulates flow estimation as the minimization of an enegy function that combines the brightness constancy constraint and a smoothness term. While robust to moredate noise and capable of estimating full dense flow, the method can oversmooth motion boundaries and struggles with large displacements.

- **TV-L1 method [30]:**

  This approach refines the variational method of Horn and Schunck by introducing a total variation (TV) regularization term and an L1 norm for the data term. The resulting algorithm is more robust and was implemented in the previous work of Bauer [15]. Nevertheless, it is computationally expensive and may not be the best choice for real-time applications.

- **Farneback method [31]:** Farneback introduced a dense optical flow method based on local polynomial expansions. The idea is to approximate the neighborhood of each pixel ith a quadratic polynomial and then estimate how the polynomials change between frames.

  This method offers a good trade-off between accuracy and computational efficiency. When accelerated using GPUs (e.g. via OpenCV CUDA modules), it can achieve real-time performance even on high-resolution images.

In this work, the Farneback method has been selected due to its computational efficiency, robustness for medium-range motions, and the availability of an optimized CUDA implementation.

3.3.3 Farneback algorithm

The Farneback optical flow method is a dense flow estimation algorithm based on polynomial expansion of local image neighborhoods. Its core idea is to approximate the intensity variation in a small image region using a quadratic polynomial, and then infer the motion between frames by analyzing how these polynomials deform over time.

The first step is to approximate the image intensity function $I(x, y)$ in a local neighborhood using a second-order Taylor expansion:

$$I(x, y) \approx x^T A x + b^T x + c \tag{3.7}$$

where:

- $x = (x, y)^T$ represents the spatial coordinates,
- A is a symmetric 2×2 matrix encoding the second-order spatial derivatives,
- b is a 2×1 vector representing the first-order spatial derivatives,
- c is a scalar term corresponding to the local intensity offset.

This local polynomial approximation can be efficiently computed over the entire image using Gaussian-weighted least squares fitting, a process known as polynomial expansion.

When the scene undergoes a pure translation $d = (d_x, d_y)^T$ between two consecutive frames, the transformed intensity function $I'(x)$ can be related to the original by:

$$I'(x) = I(x - d) \tag{3.8}$$

Substituting the polynomial approximation yields:

$$I'(x) \approx (x - d)^T A (x - d) + b^T (x - d) + c \tag{3.9}$$

Expanding the right-hand side leads to:

$$I'(x) \approx x^T A x + (b - 2Ad)^T x + (d^T A d - b^T d + c) \tag{3.10}$$

Comparing terms, we observe that:

$$A' = A \tag{3.11}$$
$$b' = b - 2Ad \tag{3.12}$$
$$c' = d^T A d - b^T d + c \tag{3.13}$$

where A' and b' represent the new polynomial coefficients after displacement.

Thus, the displacement d can be estimated by solving the linear system:

$$2Ad = b - b' \tag{3.14}$$

assuming that A is non-singular and well-conditioned.

In practice, direct estimation is complicated by noise, local deformations, and the limited validity of polynomial approximations. To address these issues, the following refinements are applied:

- Estimate the displacement over a local window using Gaussian weighting to prioritize central pixels.
- Aggregate local estimates using weighted least squares to improve robustness against noise.
- Employ a multi-scale coarse-to-fine pyramid approach to capture both small and large displacements effectively.

Coarse-to-fine pyramid approach

Large displacements between frames can cause local matching failures if only a single resolution is used. To handle this, the Farneback method employs a coarse-to-fine multi-scale pyramid strategy [32].

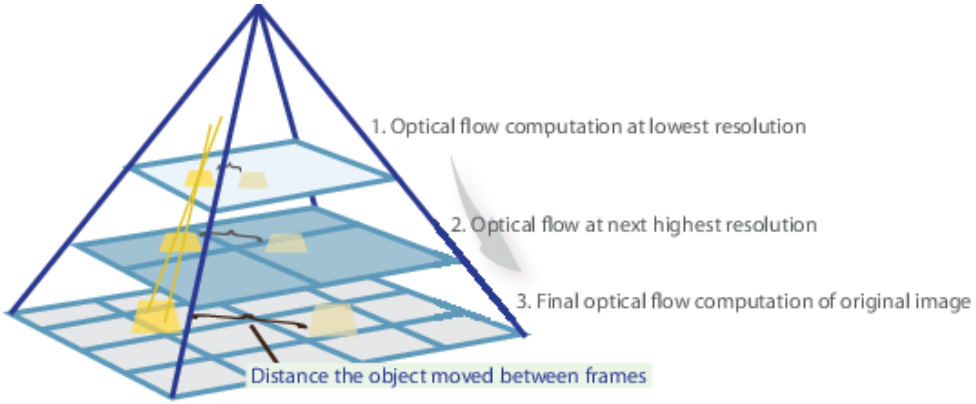


Figure 3.9: Coarse-to-fine pyramid approach. Source: Matlab

In the pyramid approach:

- The original images are repeatedly downsampled by a scaling factor (typically 0.5), producing several lower-resolution versions.
- Optical flow is first estimated at the coarsest (smallest) scale, where large motions appear reduced and easier to estimate.
- The initial flow estimate is then propagated to the next finer scale, where it is refined using the higher-resolution image data.
- This process continues until the finest scale (the original resolution) is reached, producing a dense and detailed optical flow field.

The pyramid-based coarse-to-fine strategy is crucial for ensuring that large motions do not overwhelm the local polynomial approximation, maintaining both the accuracy and robustness of the Farneback method.

This hierarchical refinement significantly enhances the capability of the algorithm to deal with large inter-frame displacements, dynamic scenes, and varying object velocities.

3.3.4 Implementation

This algorithm has been implemented in a ROS2 package called `optical_flow_computation`. This package contains a node responsible for computing the optical flow between two consecutive images. The node subscribes to two topics: the first for the left image and the second for the right image. Once it receives both images, it computes the optical flow and publishes the result on an output topic.

Regarding the algorithm implementation, the `cv::cuda::FarnebackOpticalFlow` class from OpenCV was used. This class provides a CUDA-accelerated version of the Farneback algorithm, allowing computations to be performed directly on the GPU. The OpenCV implementation of the Farneback algorithm is highly optimized and designed to leverage the parallel architecture of modern GPUs, significantly reducing execution time compared to CPU-based methods. This class is included in the header `opencv2/cudaoptflow.hpp`, which generally requires manually building OpenCV with CUDA support.

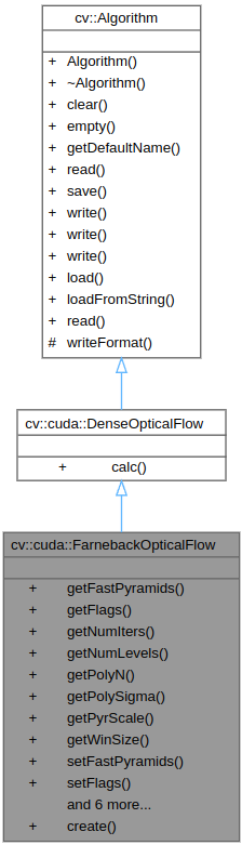


Figure 3.10: CUDA Farneback class. Source: OpenCV

To facilitate tuning and configuration, the following ROS2 parameters were added to the node:

Table 3.1: ROS2 parameters for the optical flow computation node

ROS2 parameter	Definition
image_topic	Input topic name for raw RGB images used in optical flow estimation
optical_flow_topic	Output topic name where the computed optical flow is published
pyr_scale	Scaling factor between pyramid levels
levels	Number of pyramid layers, including the original image
winsize	Averaging window size for smoothing derivatives; larger values improve robustness but may oversmooth motion boundaries
iterations	Number of iterations at each pyramid level; higher values improve convergence in complex motion areas
poly_n	Size of the pixel neighborhood used for polynomial expansion
poly_sigma	Standard deviation of the Gaussian used to smooth derivatives before polynomial expansion
flags	Operation flags passed to the Farneback function
use_sim_time	Boolean indicating whether to use simulation time (`/clock`) instead of system time

The node can be added to a launch file as shown in the following example:

Code 3.2: Optical flow computation launch

```
launch_ros.actions.Node(
            package='optical_flow_computation',
            executable='optical_flow_node',
            output='screen',
            parameters=[
              {"image_topic": input_gray_left_topic},
              {"optical_flow_topic": "perception_pipeline/
    optical_flow"},
              {"pyr_scale": 0.5},
              {"levels": 3},
              {"winsize": 15},
              {"iterations": 3},
              {"poly_n": 5},
              {"poly_sigma": 1.2},
              {"flags": 0},
              {"use_sim_time": use_sim_time}
              ],
          ),
```

The output image is published on the topic `/perception_pipeline/optical_flow`. This image is published as a `sensor_msgs/Image` message with a `32FC2` encoding format, representing a 32-bit floating-point optical flow image with two channels. The first channel corresponds to the displacement along the x-axis, and the second channel corresponds to the displacement along the y-axis.

In addition to the flow image, two debug images are published on the `/debug` topic to facilitate debugging and result visualization. One debug image shows the original image with optical flow vectors superimposed, and the other displays an RGB image where the intensity represents the magnitude of the optical flow vector and the color represents its direction.

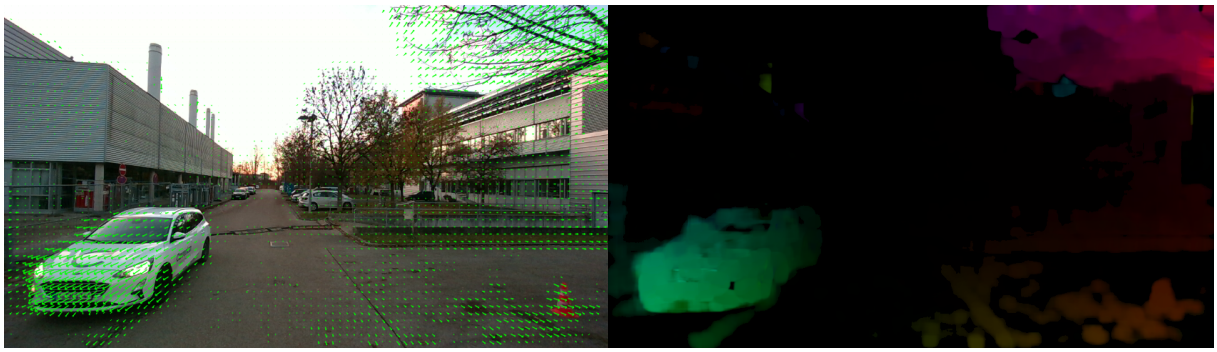


Figure 3.11: Optical flow debug images. Left: Optical flow vectors superimposed on the original image. Right: RGB image where the intensity represents the magnitude and the color represents the direction of the optical flow vector. Source: own work

These additional debug outputs greatly facilitate the qualitative evaluation of the optical flow results and enable faster identification of potential errors during the development and testing phases.

3.4 Perception pipeline: Ego motion estimation

In the context of autonomous and automated vehicles, estimating the ego motion is crucial to understand how the vehicle is moving in the environment. In the scope of this work, ego motion is defined as the relative motion of the vehicle with respect to the environment. Specifically, the 3D position of the camera in one frame, relative to the previous frame.

This information is essential for the Kalman Filter step where the 6D field will be estimated, since the optical flow output is relative to the camera frame, and the Kalman Filter requires knowledge of the camera's position in the world frame to properly compensate for its motion.

For this purpose, visual odometry (VO) techniques are used to estimate the required information using the stereo setup input. Incorporating the depth information from the stereo camera allows the direct recovery of metric information, providing robustness and accuracy.

This section describes the concept behind visual odometry, the coordinate systems used, the specific algorithmic approach that was implemented, as well as the ROS2 implementation details.

3.4.1 Visual odometry

Visual odometry (VO) refers to the process of estimating the motion of a camera by analyzing a sequence of images it captures. Originally introduced by Moravec [33] and later formalized by Nistér et al. [34], VO has become a fundamental technique for mobile robotics and autonomous driving [35].

The core idea behind VO is to track visual features across consecutive frames and infer the relative motion of the camera from these observations. The estimated motion provides a continuous estimate of the vehicle trajectory without relying on external localization systems.

Visual odometry methods are generally categorized based on the type of input and processing approach used:

- **Monocular VO:** Utilizes a single camera as input. It is cost-effective but suffers from scale ambiguity, meaning that the absolute scale of the motion (e.g., in meters) cannot be determined without additional information. Monocular VO is often used in conjunction with other sensors (e.g., IMU) to resolve this ambiguity [35].
- **Stereo VO:** Used two synchronized cameras to estimate depth through triangulation, which allows direct recovery of metric scale. This removes the scale ambiguity present in monocular systems, producing more accurate and robust motion estimation. Stereo VO is especially beneficial in environments where monocular methods struggle, such as in low-texture areas or during large baseline movements, thanks to the additional geometric constraints provided by the known baseline distance between the cameras [35].

Processing approaches are typically divided into two categories:

- **Feature-based methods:** Extract and match features (e.g., corners, edges) across frames, estimating motion from the displacement of these features.
- **Direct methods:** Directly use pixel intensities to estimate motion without explicit feature extraction.

Recently, **deep learning-based visual odometry** approaches have emerged, combining different types neural networks to estimate motion directly from image sequences. These methods aim to improve robustness under challenging conditions, such as illumination changes and textureless environments [35].

This work adopts a **Stereo VO:** and **stereo feature-based** since the depth information is already available from the stereo Camera and using feature matching to estimate motion robustly. This choice provides a good balance between accuracy and computational efficiency, making it suitable for real-time applications

3.4.2 Coordinate systems

To properly understand the visual odometry algorithm and this work, it is important to define the coordinate systems used in the implementation. The following coordinate frames are defined in the context of this project:

- **camera_optical_frame** The optical coordinate frame attached to the camera at the current time step. All of the optical frames follow the ROS2 convention, where the z-axis points forward, the x-axis points to the right of the camera, and the y-axis points downwards.
- **camera_optical_frame_previous** The optical coordinate frame attached to the camera at the previous time step of the visual odometry computation.
- **camera_optical_frame_initial** The coordinate frame attached to the camera at the start of the sequence.
- **world** Coincident with camera camera_optical_frame_initial but rotated. Z-axis points up, x-axis points forward, and y-axis points to the left.

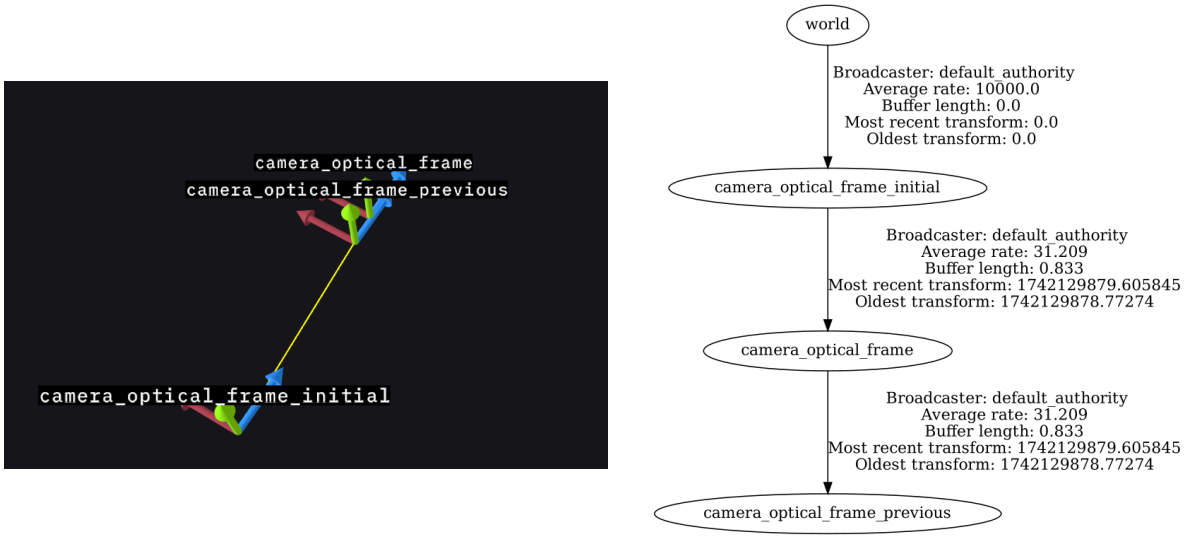


Figure 3.12: Coordinate frames and ROS2 TF tee. Left: Coordinate frames visualized using Foxglove. Right: ROS2 Transform Tree of the project generated using tf2_tools. Source: own work

Visual odometry algorithm

The visual odometry algorithm implemented in this work follows a stereo vo and feature-based stereo approach. It estimates the relative motion of the camera by detecting and matching visual features between consecutive frames, reconstructing their 3D positions using the depth image, and solving the camera pose using a Perspective-n-Point (PnP) algorithm [36] with RANSAC for outlier rejection.

A schematic overview of the algorithm steps is presented in Algorithm 1.

Algorithm 1 Stereo Visual Odometry Pipeline

Require: RGB images I_{k-1}, I_k, and depth image D_{k-1}
Ensure: Estimated relative pose (R, t) between frames

1: Detect ORB features in I_{k-1} and I_k using GPU acceleration
2: Match ORB features between I_{k-1} and I_k
3: Reproject matched keypoints from I_{k-1} into 3D using D_{k-1}
4: Solve PnP with RANSAC using 3D-2D correspondences
5: Refine the pose estimation with standard solvePnP using inliers
6: Estimate covariance based on reprojection residuals
7: Apply statistical filtering and optional exponential smoothing
8: Output (R, t), covariance matrix, and debug images

The code of the algorithm is implemented without any ROS2 dependencies, allowing it to be easily reused in other projects. In the next sections, the main steps of the algorithm are described in detail.

Feature detection and matching

Feature detection and matching constitute the first step in the visual odometry pipeline. In this phase, some points, that will serve to compute the desired transformation, will be detected in two consequtive frames. For this, the ORB (Oriented FAST and Rotated BRIEF) [37] algorithm will be used.

ORB is a binary feature descriptor that builds upon two classical methods: FAST (Features from Accelerated Segment Test) [38] for keypoint detection and BRIEF (Binary Robust Independent Elementary Features) [39] for descriptor computation.

In this context, a keypoint is a point in the image that is distinctive and repeatable (typically corners or blobs), that can be reliably identified across multiple frames. Around each keypoint, a descriptor is computed: a compact representation of the visual appearance in its local neighborhood. These descriptors allow the system to match corresponding features between frames, even under variations in viewpoint.

FAST is a corner keypoint detection algorithm that quickly identifies points in the image with significant intensity change compared to their surroundings. BRIEF, on the other hand, generates a binary string descriptor by comparing the intensity of random pairs of pixels in a small patch around the keypoint.

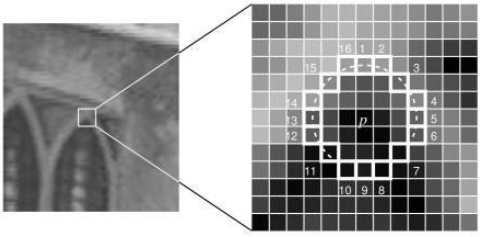


Figure 3.13: FAST algorithm. Source: [40]

ORB enhances these methods by introducing rotation and scale invariance. The orientation of each keypoint is computed to make BRIEF descriptors rotation-invariant, and a multi-scale pyramid is used to handle changes in scale.

The feature extraction and matching process is performed on the GPU using OpenCV's CUDA modules. This allows real-time processing even at high image resolutions.

The steps involved are:

1. Detect ORB features separately in the previous and current grayscale images.
2. Compute binary descriptors for each detected feature.
3. Match descriptors between the previous and current frames using brute-force Hamming distance.
4. Extract the coordinates of matched keypoints for subsequent motion estimation.

3D-2D correspondences

Once the feature matching step is completed, the next task is to establish 3D-2D correspondences between the previous and current frames. This is necessary for estimating the camera motion through Perspective-n-Point (PnP) techniques.

For each matched keypoint in the previous frame, the corresponding depth value is retrieved from the depth image associated with that frame. If the depth value is within a valid range, the 2D keypoint is reprojected into a 3D point using the intrinsic parameters of the stereo camera.

The reprojection follows the standard pinhole camera model:

$$X = (u - c_x) \cdot \frac{d}{f_x}, \quad Y = (v - c_y) \cdot \frac{d}{f_y}, \quad Z = d \tag{3.15}$$

where (u, v) are the pixel coordinates of the keypoint, (c_x, c_y) are the coordinates of the principal point, (f_x, f_y) are the focal lengths in pixels, and d is the depth value.

This process results in a set of valid 3D points extracted from the previous frame, and their corresponding 2D projections detected in the current frame. These 3D-2D pairs are later used to estimate the relative pose of the camera.

If the depth value is invalid (e.g., missing or outside the valid range), the corresponding match is discarded to avoid introducing noise into the motion estimation process. In the followign figure, a set of detected keypoint can be seen. The ones with valid depth are shown in green, while the ones with invalid depth are shown in red.

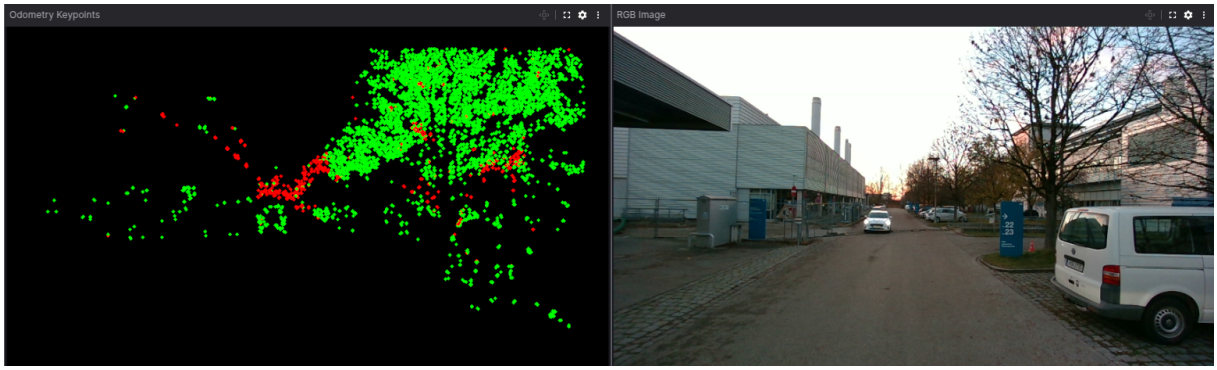


Figure 3.14: Odometry Keypoints with depth filtering and RGB Image. Source: own work

Motion estimation using PnP and RANSAC

Once 3D-2D correspondences are available, the relative camera motion between two frames can be estimated by solving the Perspective-n-Point (PnP) problem [36]. The goal is to compute the rotation $\mathbf{R}$ and translation $\mathbf{t}$ that map the 3D points observed in the previous frame to their 2D projections in the current image.

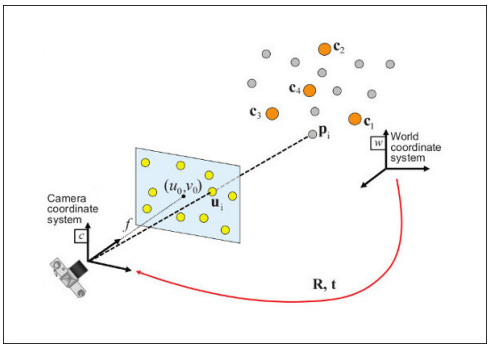


Figure 3.15: Perspectective-n-Point (PnP) problem. Source: [41]

Given a set of n correspondences $\{(\mathbf{X}_i, \mathbf{x}_i)\}$, where $\mathbf{X}_i$ are 3D points in the previous frame and $\mathbf{x}_i$ their 2D image projections in the current frame, PnP finds a transformation that minimizes the reprojection error under the pinhole camera model:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \mathbf{K}\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{3.16}$$

Here, $\mathbf{K}$ is the camera intrinsic matrix, and $\mathbf{R}, \mathbf{t}$ represent the relative pose of the current frame with respect to the previous one. The symbol $\sim$ denotes equality up to scale in homogeneous coordinates.

In practice, 3D-2D correspondences may include mismatches or noisy observations. To robustly estimate the pose in the presence of such outliers, RANSAC (Random Sample Consensus) is used in conjunction with PnP.

RANSAC operates by randomly selecting minimal subsets of correspondences, solving the PnP problem for each subset, and evaluating how many correspondences agree with the estimated pose. The solution with the highest number of inliers is selected, and the pose is then refined using all inliers to minimize the overall reprojection error.

In this work, the function `cv::solvePnPRansac` from OpenCV is used to perform motion estimation. This function takes as input the 3D points from the previous frame, their 2D projections in the current frame, and the camera intrinsics, and returns the estimated rotation and translation vectors.

The steps involved are:

1. Project 3D points from the previous frame into the current image using candidate pose.
2. Measure the reprojection error against actual 2D keypoints.
3. Select inliers whose error is below a given threshold.
4. Repeat for several random subsets and select the best solution.
5. Refine the pose using all inliers.

The resulting output consists of:

- `rvec` a rotation vector in axis-angle form.
- `tvec` a 3D translation vector.

The `rvec` returned by `solvePnPRansac` is a Rodrigues rotation vector or axis-angle rotation vector, a compact 3×1 representation of rotation. The direction of the vector indicates the axis of rotation, while its magnitude represents the rotation angle in radians around that axis. This vector can be converted to a rotation matrix using the Rodrigues formula.

This matrix describes the transformation from the previous camera frame to the current one, and can be used for pose chaining, trajectory estimation, or visual odometry back-end integration.

Covariance estimation

After estimating the camera pose using PnP and RANSAC, it is often useful to quantify the uncertainty associated with this estimate. This is particularly relevant when integrating the pose into filtering frameworks, such as an Extended Kalman Filter (EKF), where a covariance is required.

In this work, the pose covariance is approximated based on the reprojection residuals of the inlier correspondences identified by RANSAC. Once the best pose has been selected, all 3D points are projected onto the image using the estimated transform, and the residual reprojection errors are computed as:

$$\mathbf{e}_i = \mathbf{x}_i^{\text{observed}} - \mathbf{x}_i^{\text{projected}}(R, t) \tag{3.17}$$

The residuals are stacked and used to compute a sample covariance matrix:

$$\Sigma = \frac{1}{N-1} \sum_{i=1}^{N} (\mathbf{e}_i - \bar{\mathbf{e}})(\mathbf{e}_i - \bar{\mathbf{e}})^T \tag{3.18}$$

Although this approach does not yield an exact pose covariance, it provides a reasonable and fast approximation that reflects the quality of the inlier set and the geometric configuration of the matched points.

Motion filtering and smoothing

The pose estimates obtained via PnP are subject to noise due to imperfect keypoint matching, depth errors, or small instabilities in the feature geometry. To improve the consistency of the estimated trajectory, a filtering and smoothing step is applied. Two approaches are combined:

- **Statistical filtering** To detect and remove outliers in both translation and rotation components based on deviation from a local history.
- **Exponential smoothing** To smooth the motion estimate over time using a weighted moving average.

Every component of the translation and rotation vector is separately. A sliding window is stored for each component to compute its mean and standard deviation. New values that exceed a fixed threshold or deviate significantly from the mean are replaced with the mean.

Once filtered, the values are smoothed using exponential smoothing:

$$s_t = \alpha x_t + (1-\alpha)s_{t-1} \tag{3.19}$$

where s_t is the smoothed value at time t, x_t is the current value, and α is a smoothing factor between 0 and 1.

Rotation vectors are smoothed component-wise in Rodrigues form. Although this is a simplification compared to quaternion-based smoothing on $SO(3)$, it performs well in practice for small motions.

After filtering and smoothing, the pose is converted to a 4x4 transformation matrix and stored as the final motion estimate for the current frame.

3.4.3 ROS2 Implementation

The previous algorithm was implemented in c++ class called `VisualOdometry`. This class contains all the methods and attributes required to perform the visual odometry algorithm. It is used in a package called `visual_odometry` that performs the ego motion estimation.

The ROS2 node subscribes to the depth image topic, the camera info topic that contains the camera intrinsic parameters, and the color image topic. It uses the VisualOdometry class to compute the rvec and tvec transformation as well as the covariance matrix and the debug image (figure 3.14).

Once the computation is finished, the node publishes the results on the following topics:

- `/perception_pipeline/odom` Odometry topic with the camera pose in the camera initial frame.
- `/perception_pipeline/camera_frame_tf` Camera frame transform topic with the camera pose in the previous frame relative to the current frame. It is published as a separate topic because in the Kalman filter steo, this information is required with an exact frame syncronization.
- `/perception_pipeline/path` Path topic with the camera pose in the world frame (figure 3.16). This topic is used to visualize the camera trajectory.
- `/debug/odometry_keypoints` Debug topic with the detected keypoint. In red the keypoints with invalid depth and in green the keypoints with valid depth.
- `/tf` The camera frame transform is published as a ROS2 transform. This allows to visualize the camera pose in the world frame using tools like RVIZ or Foxglove.

Table 3.2: ROS 2 parameters used in visual_odometry package

ROS2 Parameter	Definition
color_image_topic	Topic name for the input color image.
depth_image_topic	Topic name for the input depth image.
camera_info_topic	Topic providing intrinsic camera parameters.
odometry_debug_topic	Topic where debug keypoints for odometry are published.
camera_frame_tf_topic	Topic containing camera pose as a transform (TF).
max_depth_odom	Maximum valid depth (in meters) used for odometry.
min_depth_odom	Minimum valid depth (in meters) used for odometry.
odometry_debug_image	Enables publishing of debug images with tracked keypoints.
publish_odom	Enables publishing of odometry estimates.
odom_topic	Topic where odometry data is published.
publish_path	Enables publishing of the full trajectory path.
path_topic	Topic where the estimated path is published.
use_sim_time	Uses simulation time instead of system time (for Gazebo, etc.).

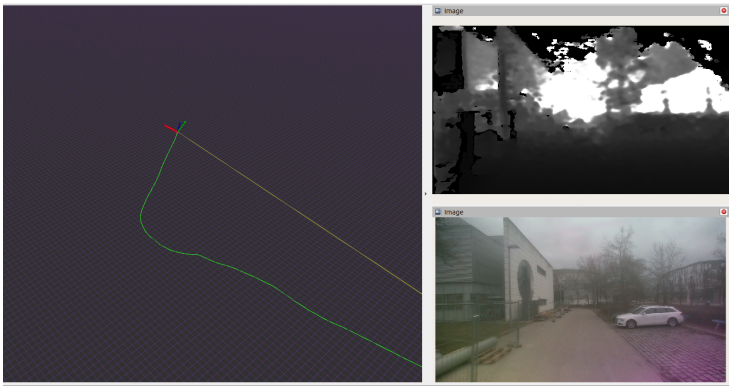


Figure 3.16: Path and depth camera topics. Source: own work

To configure the node, the following ROS2 parameters were added:

3.5 Perception pipeline: Kalman filter

After introducing the modules responsible for the depth computation (4.3.1), optical flow computation (4.3.2), and ego-motion estimation (4.3.3), this section addresses the core component of the perception pipeline: the estimation of a dense six-dimensional field using a Kalman filter.

The objective of this stage is to compute, for a subset of selected pixels of the image, both their 3D position and 3D velocity over time. This results in a per-pixel six-dimensional vector that captures the spatial and dynamic state of the scene.

This idea aligns with the concept of 6D vision (see Section 2.2), which was initially explored at the FTM Chair by Bauer [15] and later validated in a ROS1 implementation by Pastor [17].

Each point is represented by a `WorldPoint`, which maintains its own Kalman filter and a 6D state vector composed of 3D position and velocity in world coordinates. At each timestep, the point's state is updated using three main sources of input:

- Optical flow, providing pixel-level motion information between frames.
- Depth data, used to reconstruct the 3D location of the point via reprojection.
- Ego-motion, which accounts for the movement of the camera relative to the environment.

The Kalman filter follows a standard constant-velocity model in 3D space. Predicted states are projected back into the image frame for association with new measurements, producing real-time filtered estimates of position and velocity at pixel level resolution.

This approach differs from object-based or grid-based tracking by focusing on local, independent point estimation. Points are initialized or removed based on visibility, depth consistency, and motion reliability. A regular grid helps initialize new points, and the use of OpenMP [42] allows for paralellizing the updates of all active filters tracked points, leading to a significant speedup in the computation.

The system architecture separates the Kalman logic into a core module that operates independently of ROS, and a ROS2 node that manages communication, timing, and visualization. The output includes a 6D image representation, debug data, and visualization markers for use in tools such as RViz and Foxglove.

3.5.1 Kalman filter algorithm

In this module, each selected pixel or `WorldPoint` is associated with its own Kalman Filter instance that estimates their local state. The filters operate independently and are updated at each timestep using new input from the previous perception modules. This approach allows the system to maintain a dense spatial resolution while exploit paralell processing capabilities.

The Kalman Filter, first introduced by R.E. Kalman in 1960 [43], is a recursive estimation algorithm, designed for linear, discrete-time dynamical systems with Gaussian noise. It estimates the internal state of a system by predicting its evolution over time and correcting the prediction using new measurements. Taking this into account, the first thing that is necessary to implement a Kalman filter is to have the called **system model** and the **measurement model**.

The system model is a state space representation of the system, which describes how the state evolves over time. It is defined by the following equations:

$$\mathbf{x}_k = \mathbf{A} \cdot \mathbf{x}_{k-1} + \mathbf{B} \cdot \mathbf{u}_k + \boldsymbol{\omega}_k \tag{3.20}$$

Here, $\mathbf{x}_k$ is the state vector at time k, $\mathbf{A}$ is the state transition matrix, $\mathbf{u}_k$ is the control input, $\mathbf{B}$ is the control input matrix, and $\boldsymbol{\omega}_k$ is the process noise, assumed to be zero-mean Gaussian with covariance $\mathbf{Q}$:

$$\boldsymbol{\omega}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \tag{3.21}$$

The measurement model describes how the measurements are related to the state. It is defined by the following equations:

$$\mathbf{z}_k = \mathbf{H} \cdot \mathbf{x}_k + \boldsymbol{\nu}_k \tag{3.22}$$

where $\mathbf{z}_k$ is the measurement vector, $\mathbf{H}$ is the measurement matrix, and $\boldsymbol{\nu}_k$ is the measurement noise, assumed to be zero-mean Gaussian with covariance $\mathbf{R}$:

$$\boldsymbol{\nu}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}) \tag{3.23}$$

When both models are linear, as in the equations above, the standard Kalman filter can be used. However, if either model is nonlinear, the Extended Kalman Filter (EKF) [44] must be applied. The EKF approximates nonlinear functions by linearizing them around the current state estimate using the Jacobian (first order Taylor series approximation).

Standard Kalman filter

The standard Kalman filter operates in two main phases at each time step: prediction and correction. Each phase consists of a sequence of matrix operations that update both the estimated state and its associated covariance based on the system and measurement models.

Prediction step The prediction step starts by propagating the previous filtered state $\mathbf{x}_{k-1}$ forward in time using the system model. This results in the predicted state:

$$\hat{\mathbf{x}}_k = \mathbf{A} \cdot \mathbf{x}_{k-1} + \mathbf{B} \cdot \mathbf{u}_k \tag{3.24}$$

where $\mathbf{A}$ is the state transition matrix, $\mathbf{B}$ the control input matrix, and $\mathbf{u}_k$ the control vector at time k. The predicted covariance $\hat{\mathbf{P}}_k$ is then computed as:

$$\hat{\mathbf{P}}_k = \mathbf{A} \cdot \mathbf{P}_{k-1} \cdot \mathbf{A}^\top + \mathbf{Q} \tag{3.25}$$

where $\mathbf{P}_{k-1}$ is the previous covariance and $\mathbf{Q}$ is the process noise covariance.

Correction step Once the state and covariance have been predicted, the correction step begins and the expected measurement is computed using the measurement model:

$$\hat{\mathbf{z}}_k = \mathbf{H} \cdot \hat{\mathbf{x}}_k \tag{3.26}$$

The predicted measurement covariance is then:

$$\hat{\mathbf{T}}_k = \mathbf{H}_k \cdot \hat{\mathbf{P}}_k \cdot \mathbf{H}_k^\top \tag{3.27}$$

When a new measurement $\mathbf{z}_k$ is available, it is compared with the predicted measurement $\hat{\mathbf{z}}_k = \mathbf{h}(\hat{\mathbf{X}}_{w|k})$ to evaluate how consistent the prediction is with the actual observation. The difference between them is known as the innovation vector:

$$\mathbf{s}_k = \mathbf{z}_k - \hat{\mathbf{z}}_k \tag{3.28}$$

This vector represents the residual between what the filter expected to observe and what was actually measured. If the model and previous state are accurate, the innovation should be small and centered around zero. Large values in $\mathbf{s}_k$ may indicate unexpected changes in the system or incorrect predictions.

To properly weight the innovation in the update step, its uncertainty must be quantified. This is given by the innovation covariance matrix:

$$\mathbf{S}_k = \hat{\mathbf{T}}_k + \mathbf{T}_k \tag{3.29}$$

Here, $\hat{\mathbf{T}}_k$ is the predicted measurement covariance derived from the current state estimate, and $\mathbf{T}_k$ is the measurement noise covariance, defined by the sensor model. The matrix $\mathbf{S}_k$ thus quantifies the total expected uncertainty in the innovation vector, combining both predicted state uncertainty projected into measurement space and known sensor noise.

The innovation vector and its covariance are crucial for the correction step of the filter: they determine how strongly the measurement should influence the state update. A large $\mathbf{S}_k$ implies high uncertainty, which results in a smaller correction; conversely, a small $\mathbf{S}_k$ leads to a more aggressive update.

Once the innovation and its covariance are computed, the Kalman gain $\mathbf{K}_k$ is calculated. The Kalman gain is a matrix that determines how much the predicted state should be adjusted based on the new measurement. It is computed as:

$$\mathbf{K}_k = \hat{\mathbf{P}}_k \cdot \mathbf{H}^\top \cdot \mathbf{S}_k^{-1} \tag{3.30}$$

Finally, the state and covariance are updated using the Kalman gain and the innovation vector. The updated state is given by:

$$\mathbf{x}_k = \hat{\mathbf{x}}_k + \mathbf{K}_k \cdot \mathbf{s}_k \tag{3.31}$$

and the updated covariance becomes:

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \cdot \mathbf{H}) \cdot \hat{\mathbf{P}}_k \tag{3.32}$$

This completes one full iteration of the standard Kalman filter. The filtered state $\mathbf{x}_k$ and covariance $\mathbf{P}_k$ serve as the input for the next prediction step.

Extended Kalman filter

When the system or measurement model is nonlinear, the standard Kalman filter cannot be applied directly. In such cases, the Extended Kalman Filter (EKF) is used. The EKF extends the classical Kalman filter by linearizing the nonlinear functions around the current estimate using a first-order Taylor expansion.

Let $\mathbf{f}(\cdot)$ represent the nonlinear system model, and $\mathbf{h}(\cdot)$ the nonlinear measurement model. The EKF performs the prediction step by applying the nonlinear dynamics to the previous state:

$$\hat{\mathbf{x}}_k = \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k) \tag{3.33}$$

The predicted covariance is obtained using the Jacobian $\mathbf{F}_k$ of $\mathbf{f}(\cdot)$ with respect to the state:

$$\hat{\mathbf{P}}_k = \mathbf{F}_k \cdot \mathbf{P}_{k-1} \cdot \mathbf{F}_k^\top + \mathbf{Q} \tag{3.34}$$

The predicted measurement is computed by evaluating the nonlinear measurement function:

$$\hat{\mathbf{z}}_k = \mathbf{h}(\hat{\mathbf{x}}_k) \tag{3.35}$$

and its Jacobian with respect to the state is:

$$\mathbf{H}_k = \left. \frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_k} \tag{3.36}$$

The predicted measurement covariance becomes:

$$\hat{\mathbf{T}}_k = \mathbf{H}_k \cdot \hat{\mathbf{P}}_k \cdot \mathbf{H}_k^\top \tag{3.37}$$

Given a new measurement $\mathbf{z}_k$, the innovation is:

$$\mathbf{s}_k = \mathbf{z}_k - \hat{\mathbf{z}}_k \tag{3.38}$$

and the innovation covariance is:

$$\mathbf{S}_k = \hat{\mathbf{T}}_k + \mathbf{R} \tag{3.39}$$

Once the innovation and its covariance are computed, the Kalman gain is calculated as:

$$\mathbf{K}_k = \hat{\mathbf{P}}_k \cdot \mathbf{H}_k^\top \cdot \mathbf{S}_k^{-1} \tag{3.40}$$

Finally, the updated state and covariance are:

$$\mathbf{x}_k = \hat{\mathbf{x}}_k + \mathbf{K}_k \cdot \mathbf{s}_k \tag{3.41}$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \cdot \mathbf{H}_k) \cdot \hat{\mathbf{P}}_k \tag{3.42}$$

This completes one full iteration of the Extended Kalman Filter. It retains the recursive structure of the classical filter while enabling the integration of nonlinear system or measurement models by local linearization.

3.5.2 Mathematical model

Now that the Kalman filter algorithm has been introduced, we can define the mathematical model used in this work. First, the system model is defined and then the measurement model will be defined.

System model

For the system model, the state vector $\mathbf{x}_{k|w}$ is defined as a 6×1 vector that contains the position and velocity of the tracked point in world coordinates (w):

$$\mathbf{x}_{k|w} = \begin{bmatrix} X \\ Y \\ Z \\ \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix} \tag{3.43}$$

This vector is propagated over time using a constant-velocity motion model, assuming no acceleration between frames. When no ego-motion compensation is applied, the evolution of the state is defined by the following linear state model system:

$$\hat{\mathbf{x}}_{k|w} = \mathbf{A}_{k|w} \cdot \mathbf{x}_{k-1|w} + \boldsymbol{\omega}_{k|w} \tag{3.44}$$

where $\hat{\mathbf{x}}_{k|w}$ is the predicted state at time k, $\mathbf{A}_{k|w}$ is the transition matrix that models constant velocity over the time step Δt, and $\boldsymbol{\omega}_{k|w}$ is the process noise.

The state transition matrix $\mathbf{A}_{k|w}$ of the system is defined as:

$$\mathbf{A}_{k|w} = \begin{bmatrix} \mathbf{I}_3 & \Delta t \cdot \mathbf{I}_3 \\ \mathbf{0}_3 & \mathbf{I}_3 \end{bmatrix} \tag{3.45}$$

where Δt is the time elapsed between frames, representing the linear motion model. The process noise is assumed to be zero-mean Gaussian:

$$\boldsymbol{\omega}_{k|w} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_{k|w}) \tag{3.46}$$

Where the process noise covariance matrix $\mathbf{Q}_{k|w}$ captures the uncertainty introduced during the state prediction. It is derived from the continuous-time system noise model [45] assuming white noise acceleration, and discretized using standard integration over the time interval Δt. The resulting structure is:

$$\mathbf{Q}_{k|w} = \begin{pmatrix} \frac{1}{3}\Delta t^2 \cdot \text{diag}(\sigma_{\dot{X}}^2, \sigma_{\dot{Y}}^2, \sigma_{\dot{Z}}^2) & \frac{1}{2}\Delta t \cdot \text{diag}(\sigma_{\dot{X}}^2, \sigma_{\dot{Y}}^2, \sigma_{\dot{Z}}^2) \\ \frac{1}{2}\Delta t \cdot \text{diag}(\sigma_{\dot{X}}^2, \sigma_{\dot{Y}}^2, \sigma_{\dot{Z}}^2) & \text{diag}(\sigma_{\dot{X}}^2, \sigma_{\dot{Y}}^2, \sigma_{\dot{Z}}^2) \end{pmatrix} \tag{3.47}$$

This formulation assumes that velocity is affected by independent white noise processes in X, Y, and Z, and that position integrates this uncertainty over time. As a result, the top-left block of $\mathbf{Q}_{k|w}$ corresponds to accumulated uncertainty in position, and the bottom-right to direct uncertainty in velocity.

To introduce the ego-motion compensation, we have to modify the system model. For this we assume, that the WorldPoints are transformed with the rotation matrix $\mathbf{R}_k$ and the translation vector $\mathbf{t}_k$, that model the movement of the camera between two frames (3.4.2). The position and velocity components of the state vector x_k are transformed as follows:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix}_k = \mathbf{R}_k \cdot \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}_{k-1|w} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}_k \tag{3.48}$$

$$\begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix}_k = \mathbf{R}_k \cdot \begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix}_{k-1|w} \tag{3.49}$$

which leads to the following updated system state model:

$$\mathbf{x}_k = \mathbf{A}_k \cdot \mathbf{x}_{k-1} + \mathbf{u}_k + \boldsymbol{\omega}_k \tag{3.50}$$

Where the individual components are defined as:

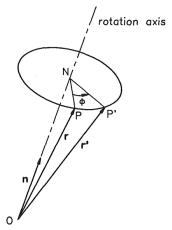


Figure 3.17: Rodrigues vector representation of the rotation matrix. Source: [46]

$$\mathbf{A}_k = \mathbf{D}_k \cdot \mathbf{A}_{k|w}, \in \mathbb{R}^{6\times 6} \tag{3.51}$$

$$\mathbf{D}_k = \begin{bmatrix} \mathbf{R}_k & \mathbf{0}_3 \\ \mathbf{0}_3 & \mathbf{R}_k \end{bmatrix} \in \mathbb{R}^{6\times 6} \tag{3.52}$$

$$\mathbf{u}_k = \begin{bmatrix} \mathbf{t}_k \\ \mathbf{0}_3 \end{bmatrix} \in \mathbb{R}^{6\times 1} \tag{3.53}$$

$$\boldsymbol{\omega}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k) \tag{3.54}$$

Now that the ego-motion was introduced in the system model, the covariace matrix $\mathbf{Q}_k$ must be defined. This matrix is not only dependant on the process noise $\sigma^2_{\dot{X}}$, $\sigma^2_{\dot{Y}}$, and $\sigma^2_{\dot{Z}}$ like the covarianze matrix without ego-motion $\mathbf{Q}_{k|w}$, but also on the ego-motion parameters. For this reason a parameter vector containing the ego-motion must be defined.

In previous works like [45], [15], and [17], the ego-motion parameters were defined by the translation components and the Euler angles:

$$\mathbf{p}_k = \begin{bmatrix} t_{k|x} & t_{k|y} & t_{k|z} & \theta_k & \phi_k & \psi_k \end{bmatrix}^T \tag{3.55}$$

This approach has the big disadvantage that the Euler angles are not continuous and can lead to discontinuities in the covariance matrix. Moreover, its necessary to know beforehand which Euler angle convention was used to build the rotation matrix $\mathbf{R}_k$. This can lead to confusion and errors in the implementation and also limits the modularity of the code.

For this reason, in this work, the ego-motion rotation components are defined using the Rodrigues vector $\mathbf{r}_k$ that is computed from the rotation matrix $\mathbf{R}_k$:

$$\mathbf{p}_k = \begin{bmatrix} t_{k|x} & t_{k|y} & t_{k|z} & r_{k|x} & r_{k|y} & r_{k|z} \end{bmatrix}^T \tag{3.56}$$

The Euler-rodrigues vector, as introduced in the chapter 3.4.2, is a continuous representation of the rotation matrix as an vector in $\mathbb{R}^3$ where the length of the vector is the angle of rotation and the direction of the vector is the axis of rotation [46, 47] as seen in the figure 3.17. This representation is continuous and also enables the correct working of the algorithm regardless of how the rotation matrix was computed.

Now that the p_k vector is defined, we can state that the rotation matrix $\mathbf{R}_k$ and the translation vector $\mathbf{t}_k$ are build as follows:

$$\mathbf{R}_k = \mathbf{R}(\mathbf{p}_k + \zeta_k) \tag{3.57}$$
$$\mathbf{t}_k = \mathbf{t}(\mathbf{p}_k + \zeta_k) \tag{3.58}$$

Where ζ_k is the additive white noise of p_k and follows a Gaussian distribution with zero mean and covariance $\mathbf{L}$:

$$\zeta_k \sim \mathcal{N}(\mathbf{0}, \mathbf{L}), \quad \mathbf{L} = \text{diag}(\sigma^2_{t|x}, \sigma^2_{t|y}, \sigma^2_{t|z}, \sigma^2_{r|x}, \sigma^2_{r|y}, \sigma^2_{r|z}) \in \mathbb{R}^{6\times6} \tag{3.59}$$

Taking this into account, the covariance matrix $\mathbf{Q}_k$ is defined as:

$$\mathbf{Q}_k = \mathbf{D}_k \cdot \mathbf{Q}_{k|w} \cdot \mathbf{D}_k^\top + \mathbf{J}_k \cdot \mathbf{L} \cdot \mathbf{J}_k^\top \tag{3.60}$$

In the previous equation, the term J_k is the Jacobian of the state vector with respect to the ego-motion parameter vector p_k:

$$\mathbf{J}_k = \left.\frac{\partial \mathbf{x}}{\partial \mathbf{p}}\right|_{(\mathbf{x}_{k-1}, \mathbf{p}_k)} \in \mathbb{R}^{6\times6} \tag{3.61}$$

This Jacobian can be extended as follows:

$$\mathbf{J}_k = \left.\frac{\partial \mathbf{x}}{\partial \mathbf{p}}\right|_{(\mathbf{x}_{k-1}, \mathbf{p}_k)} = \begin{bmatrix} \left.\frac{\partial \mathbf{x}}{\partial \mathbf{t}_k}\right|_{(\mathbf{x}_{k-1}, \mathbf{p}_k)} & \left.\frac{\partial \mathbf{x}}{\partial \mathbf{r}_k}\right|_{(\mathbf{x}_{k-1}, \mathbf{p}_k)} \end{bmatrix} \tag{3.62}$$

The first term, the partial derivative with respect to $\mathbf{t}_k$, corresponds to the direct effect of translation on the predicted state. Since the translation vector only affects the position and not the velocity, the derivative is:

$$\frac{\partial \mathbf{x}_k}{\partial \mathbf{t}_k} = \begin{bmatrix} \mathbf{I}_3 \\ \mathbf{0}_3 \end{bmatrix} \tag{3.63}$$

The second term, the derivative with respect to the rotation vector $\mathbf{r}_k$, is more complex. The rotation matrix $\mathbf{R}_k$ is computed from $\mathbf{r}_k$ using the Euler-Rodrigues formula. In addition to the matrix itself, OpenCV provides the Jacobian $\frac{\partial \mathbf{R}_k}{\partial \mathbf{r}_k} \in \mathbb{R}^{9\times3}$, which encodes the derivative of each element of $\mathbf{R}_k$ with respect to the rotation parameters.

Let us denote the intermediate propagated state before ego-motion as:

$$\mathbf{p}_k = \mathbf{A}_{k|w} \cdot \mathbf{x}_{k-1|w} = \begin{bmatrix} \mathbf{p}_k^{\text{pos}} \\ \mathbf{p}_k^{\text{vel}} \end{bmatrix} \tag{3.64}$$

where $\mathbf{p}_k^{\text{pos}}$ and $\mathbf{p}_k^{\text{vel}}$ are the position and velocity parts of the propagated state. Then, the rotated position is given by $\mathbf{R}_k \cdot \mathbf{p}_k^{\text{pos}}$, and its derivative with respect to $\mathbf{r}_k$ is:

$$\frac{\partial(\mathbf{R}_k \cdot \mathbf{p}_k^{\text{pos}})}{\partial \mathbf{r}_k} = \sum_{i=1}^{3} \left(\frac{\partial \mathbf{R}_k}{\partial r_i} \cdot \mathbf{p}_k^{\text{pos}} \right) \tag{3.65}$$

The same procedure applies to the velocity component. The full Jacobian is then:

$$\frac{\partial \mathbf{x}_k}{\partial \mathbf{r}_k} = \begin{bmatrix} \frac{\partial(\mathbf{R}_k \cdot \mathbf{p}_k^{\text{pos}})}{\partial \mathbf{r}_k} \\ \frac{\partial(\mathbf{R}_k \cdot \mathbf{p}_k^{\text{vel}})}{\partial \mathbf{r}_k} \end{bmatrix} \in \mathbb{R}^{6\times 3} \tag{3.66}$$

Combining both components, the full Jacobian with respect to the motion parameters is:

$$\mathbf{J}_{k|w} = \begin{bmatrix} \mathbf{I}_3 & \frac{\partial(\mathbf{R}_k \cdot \mathbf{p}_k^{\text{pos}})}{\partial \mathbf{r}_k} \\ \mathbf{0}_3 & \frac{\partial(\mathbf{R}_k \cdot \mathbf{p}_k^{\text{vel}})}{\partial \mathbf{r}_k} \end{bmatrix} \tag{3.67}$$

This expression captures how the uncertainty in the ego-motion parameters propagates into the predicted state covariance, and allows the filter to adapt to the quality of the visual odometry estimates in a principled way. The Jacobian $\mathbf{J}_k$ is re-evaluated at each time step based on the current predicted state and ego-motion estimate.

Now that the full state prediction model—including ego-motion and its uncertainty—has been defined, we turn our attention to the measurement model, which relates the predicted state to the observations available from the stereo camera and optical flow pipeline.

Measurement model

The measurement model relates the predicted state to the observed measurements. In this case, the measurements are obtained from the stereo camera and the optical flow computation. The measurements are defined as a 3×1 vector:

$$\mathbf{z}_k = \begin{bmatrix} u_k \\ v_k \\ d_k \end{bmatrix} \tag{3.68}$$

where u_k and v_k are the pixel coordinates of the tracked point in the image, and d_k is the depth. In this project, the depth was considered for the measurment model, instead of the disparity like the previous works, since most of the modern stereo cameras provide the depth directly as output.

Unlike the system model, the measurement model is nonlinear. The relationship between the predicted state and the measurements is given by the pinhole camera model, which maps the 3D world coordinates to 2D pixel coordinates. The measurement model is defined as:

$$\mathbf{z}_k = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 \end{bmatrix} \cdot \frac{1}{w} \cdot \mathbf{\Pi} \cdot \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} \tag{3.69}$$

where $\mathbf{\Pi}$ is the intrinsic projection matrix of the camera, defined as:

$$\mathbf{\Pi} = \begin{bmatrix} f_x & 0 & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{3.70}$$

This projection transforms a 3D point in camera coordinates into homogeneous pixel coordinates. The final measured pixel location is obtained by normalizing the first two components by the third (perspective division), and taking the depth Z directly from the position vector. Therefore, the nonlinear measurement function $\mathbf{h}(\mathbf{x}_k)$ is defined as:

$$\mathbf{h}(\mathbf{x}_k) = \begin{bmatrix} \frac{f_x X}{Z} + c_x \\ \frac{f_y Y}{Z} + c_y \\ Z \end{bmatrix} \tag{3.71}$$

This maps the predicted 3D point to the expected image coordinates (u, v) and depth $d = Z$. Since this model is nonlinear, the Extended Kalman Filter is used. To apply the correction step of the EKF, the Jacobian of the measurement model with respect to the state vector is required.

$$\mathbf{H}_k = \left. \frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_k} \tag{3.72}$$

Only the position part of the state vector affects the measurement, so the Jacobian matrix $\mathbf{H}_k$ has non-zero entries only in the first three columns. The matrix $\mathbf{H}_k \in \mathbb{R}^{3\times 6}$ is computed as:

$$\mathbf{H}_k = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} & 0 & 0 & 0 \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \tag{3.73}$$

This Jacobian is evaluated at the predicted state $\hat{\mathbf{x}}_k$ and used in the computation of the innovation covariance.

3.5.3 Implementation

Regarding the implementation of the filter, that will later be used by the corresponding `KalmanFilterNode` ROS2 node, it is separated into two main classes as introduces before:

- `WorldPoint`: This class represents a single tracked point in the scene and manages its own Kalman filter.
- `KalmanCore`: This class manages the list of all active filters and handles their initialization and updates.

In this subsection, we will describe the high level software architecture of both of them.

WorldPoint class

The `WorldPoint` class is the core of the Kalman filter implementation. It encapsulates all the logic for managing a single tracked point in the scene, including its Kalman filter, state prediction, measurement update, and error code handling. In the following figure 3.18 the class diagram of the `WorldPoint` class is shown.

Figure 3.18: WorldPoint class. Source: own work

The initialization takes place through the method `initKalmanFilter()`, which takes a pixel location and a depth value, reprojects it to 3D world coordinates, and initializes the state vector and covariance. A regular grid ensures that only one point is initialized per cell, maintaining a uniform spatial distribution as will be explained in the following section.

The main function for updating each filter is `computeKalmanStep()`. It takes as input the depth image, optical flow, and the current ego-motion estimation. It begins by updating the measurement vector using optical flow, then retrieves the new depth from the depth image. If the new pixel lies outside the image boundaries or the depth is invalid, the function returns an error. Otherwise, it proceeds to the prediction step.

Algorithm 2 Kalman Filter Step (WorldPoint)

```
Get optical flow at previous pixel position
Compute new pixel and read new depth
if new pixel is outside bounds or depth is invalid then
    return Error
end if
Predict new state using motion model
if using ego-motion then
    Apply rotation and translation
    Compute updated uncertainty using Jacobians
end if
Project predicted state into image
Compute innovation vector and covariance
if 3-sigma test fails then
    return Error
end if
Compute Kalman gain
Update state and covariance
return Success
```

If ego-motion compensation is enabled, the predicted state is rotated and translated based on the estimated motion of the camera. Additionally, the associated uncertainty is propagated using Jacobians of the rotation matrix with respect to the Rodrigues vector. These Jacobians are computed using OpenCV, and allow precise propagation of uncertainty due to camera motion.

After predicting the state, the measurement is predicted by projecting the 3D point into the image using the pinhole camera model. This expected pixel and depth are then compared to the actual measurement, and an innovation vector is calculated.

Before applying the correction step, a 3-sigma test is used to validate the measurement. This test ensures that the observed measurement is statistically consistent with the predicted measurement given the uncertainty. Specifically, it checks whether the Mahalanobis distance of the innovation vector $\mathbf{s}_k$ is below a threshold:

$$\mathbf{s}_k^\top \cdot \mathbf{S}_k^{-1} \cdot \mathbf{s}_k < \tau \tag{3.74}$$

where $\mathbf{S}_k$ is the innovation covariance and τ is the threshold corresponding to a 99.7% confidence interval in a chi-squared distribution with 3 degrees of freedom. If the condition is not met, the measurement is considered an outlier and the correction step is skipped to maintain filter robustness.

If the update is accepted, the Kalman gain is computed, and the state and covariance are updated accordingly. The filter also checks whether the updated 3D point lies within acceptable height bounds. If the point is too high or too low, it is discarded.

KalmanCore class

The `KalmanCore` class is responsible for managing the full set of active `WorldPoint` filters in the scene. It handles the initialization, update, removal, and memory management of each tracked point, ensuring a scalable and spatially uniform filtering process across the image.

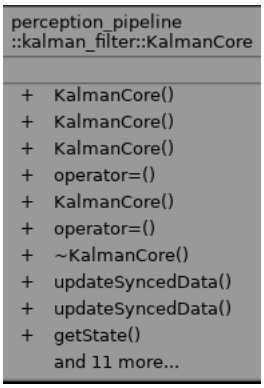


Figure 3.19: KalmanCore class.

To guarantee an even distribution of tracked points, a grid-based approach is employed (figure 3.20). The image plane is divided into a fixed number of rectangular cells, controlled by the parameters `grid_cols` and `grid_rows`. Each cell holds at most one active `WorldPoint`. This ensures that high-texture regions do not accumulate redundant points, and low-texture areas are still covered, improving robustness across the entire field of view.

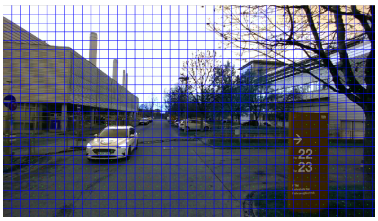


Figure 3.20: KalmanCore grid.

The core logic of each frame is executed in the `updateSyncedData()` method. This method is responsible for updating all active filters, removing invalid ones, and reinitializing new points where necessary. The complete process is shown in Algorithm 3.

Algorithm 3 Frame Processing (`KalmanCore::updateSyncedData`)

Input: depth image, optical flow, ego-motion estimate
Output: updated set of WorldPoints
for all grid cells (i, j) **do**
    if cell has an active WorldPoint **then**
        run `computeKalmanStep()` for the point
        if step fails or point outside bounds **then**
            remove WorldPoint from cell
        end if
    end if
end for
for all empty cells (i, j) **do**
    search candidate pixels in image region
    if valid depth and point found **then**
        initialize new WorldPoint and store in grid
    end if
end for

The algorithm is divided into two stages. In the first stage, each existing `WorldPoint` is updated. If the prediction or correction fails, the point is discarded. This may occur due to missing depth, invalid image location, or high innovation error detected by the three-sigma test.

In the second stage, the algorithm checks for empty grid cells. For each unoccupied cell, a new point is initialized by scanning its corresponding image region for valid pixels with usable depth. The pixel is reprojected to 3D and passed to `initKalmanFilter()` to start a new filter instance.

The grid ensures that at least one point exists per region at any time, preventing overlapping estimates and enabling scalable parallel execution. In addition, the class supports multithreading: the outer loop is parallelized with OpenMP, allowing all `WorldPoints` to be updated simultaneously. This results in a significant performance gain, especially when tracking hundreds of points across large resolutions.

3.5.4 ROS2 integration

The `KalmanFilterNode` is the central ROS 2 component that connects the Kalman filter logic with the perception pipeline. It handles all data flow by subscribing to synchronized data input streams, applying the filter update loop, and publishing processed outputs and debug information.

The node subscribes to three synchronized input topics: the depth image, the optical flow, and the ego-motion transform. Synchronization is performed using `message_filters::Synchronizer` to guarantee time alignment across all data modalities. Accurate synchronization is crucial to prevent inconsistencies in the filter prediction and correction steps.

In addition, runtime behavior is fully configurable via ROS 2 parameters. These include the grid size, filter parameters such height limits to consider, and whether ego-motion should be used. It also allows configuring all relevant noise sources via scalar parameters, including the standard deviations of optical flow, depth, process model, and ego-motion (rotational and translational) noise. These values are internally used to construct the covariance matrices required by the Kalman update equations.

The output of the node is a 2D image encoding the full 6D state field over the image plane. This image uses the `CV_32FC(7)` encoding, where each pixel contains a seven-channel float vector.

- Channels 0–2: estimated position (X, Y, Z)
- Channels 3–5: estimated velocity $(\dot{X}, \dot{Y}, \dot{Z})$
- Channel 6: binary validity flag (1.0 if valid, 0.0 if empty)

For qualitative evaluation and debugging, additional topics are published. These include:

- A RGB image with the WorldPoint positions overlaid on the original image
- A `MarkerArray` with 3D arrows, representing the estimated velocity field

The output of the Kalman Filter is shown in the figure 3.21.

Figure 3.21: Kalman Filter output. Source: own work

3.6 Perception pipeline: Object detection

The final stage of the perception pipeline is the object detection component. It is responsible for transforming the dense 6D motion field, produced by the Kalman filter, into a compact set of object-level representations. Each detected object is described by its spatial extent and estimated velocity, enabling further reasoning in downstream modules such as criticality assessment.

The object detection component performs two main tasks. First, it filters and selects reliable motion vectors from the raw state field. Then, it applies a clustering algorithm to group consistent points into distinct objects. This approach allows the system to transition from point-based to object-centric perception, which is better suited for interpreting dynamic scenes.

Like the Kalman filter module, the object detection logic is implemented in a ROS2-independent source library, and a ROS2 node handles communication and integration with the rest of the system.

3.6.1 Clustering

The goal of the clustering stage is to aggregate motion vectors into object-level groups. Each vector, defined by a 3D position and velocity, represents a localized motion hypothesis estimated by the Kalman filter. Clustering serves to identify sets of points that likely belong to the same moving object in the environment.

This task is challenging due to the sparsity and variability of the input data. Objects may vary in size, shape, and velocity; some points may be isolated or unreliable due to sensor noise or tracking loss. As a result, the clustering method must be robust, flexible, and capable of handling outliers.

Several clustering techniques have been proposed in literature for similar tasks. Among the most common are:

Add references to the clustering algorithms

- **K-Means**: a partition-based algorithm that assigns points to a fixed number of clusters. It assumes convex cluster shapes and equal density, which is often not the case in real-world perception problems.
- **Hierarchical clustering**: a tree-based method that builds a cluster hierarchy by recursively merging or splitting groups. It does not require prior knowledge of the number of clusters, but has high computational cost and is sensitive to noise.

- **Mean Shift**: a mode-seeking algorithm that clusters based on density peaks. It can discover arbitrarily shaped clusters, but tends to be computationally expensive and sensitive to bandwidth selection.
- **DBSCAN**: a density-based algorithm that identifies clusters as connected regions of high point density. It does not require the number of clusters in advance and handles noise explicitly.

Given the requirements of this project—unstructured input, unknown number of objects, and frequent outliers—DBSCAN was selected. It is well suited to segment point clouds where object boundaries are defined by density rather than geometry. Moreover, DBSCAN can be extended with a custom distance function, enabling the integration of both spatial and motion cues into the clustering process.

In this implementation, the algorithm considers two points as neighbors if they are close in space and exhibit similar velocity vectors. This joint criterion is more discriminative than position alone, especially in dynamic scenes where multiple objects may temporarily overlap in 3D space.

3.6.2 DBSCAN clustering

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a non-parametric clustering algorithm introduced by Ester et al. [48]. It defines clusters as areas of high point density and treats sparse regions as separators or noise. This density-based definition allows DBSCAN to discover clusters of arbitrary shape and to operate without requiring the number of clusters in advance.

The algorithm operates on a dataset of points and requires two parameters:

- ε: the neighborhood radius around each point
- $minPts$: the minimum number of points within the radius to form a dense region

The process begins by iterating over each point in the dataset:

1. For a given point, its ε-neighborhood is computed.
2. If the number of neighbors is greater than or equal to $minPts$, the point is marked as a *core point*.
3. A new cluster is initiated and the neighborhood of the core point is expanded recursively to include all density-connected points.
4. If a point is not a core point and not reachable from any cluster, it is marked as noise.

Two points are considered *density-connected* if there exists a chain of core points such that each is within ε of the next. This enables clusters to grow organically around dense regions and stop at natural boundaries.

Figure 3.22 illustrates the basic idea of DBSCAN. Core points have dense neighborhoods and initiate clusters. Border points lie within the neighborhood of a core point but lack enough neighbors to be core points themselves. Noise points do not belong to any cluster.

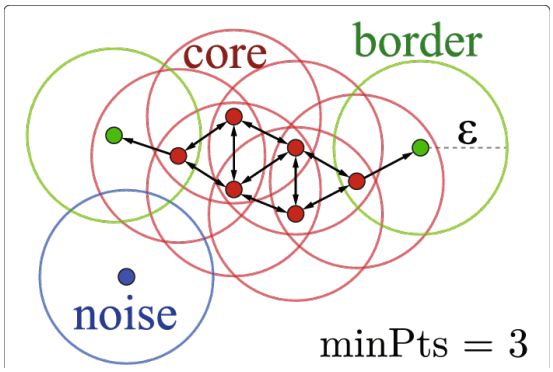


Figure 3.22: Illustration of DBSCAN clustering: core, border, and noise points. Source: [49]

This structure makes DBSCAN robust to spurious detections and suitable for real-world scenarios with irregular object shapes. In dynamic environments, it is common for moving objects to generate clusters of varying density. DBSCAN handles this without the need to fit a global model or impose strict geometric assumptions.

In our application, the algorithm is applied not in pure Euclidean space, but using a distance function that incorporates both spatial and motion similarity. The definition of this distance and its implementation are described in the following sections.

Implementation

The `ObjectDetector` class is the central component responsible for identifying dynamic objects from the 6D state field computed in the Kalman filter stage. It is designed as a self-contained module that can be reused and evaluated independently of the ROS2 framework. The main purpose of this class is to convert a dense field of motion vectors into a sparse list of detected object representations suitable for higher-level reasoning.

The class exposes a public method, `detect()`, which performs the complete processing pipeline. The input to this method is an OpenCV matrix of type `CV_32FC(7)`, where each pixel encodes a 6D motion estimate and a validity flag. The output is a list of `WorldEntity` instances, each corresponding to a cluster of consistent motion vectors.

Internally, the pipeline consists of three sequential stages. These are described below in detail.

1. CUDA-based filtering The first stage is responsible for identifying valid and informative motion vectors from the full image. Given that the input may contain a large number of pixels, including invalid or static ones, it is essential to reduce the dataset before clustering. To achieve this efficiently, a custom CUDA kernel is used. Each thread in the kernel processes one pixel, applying a sequence of checks:

- Validity: The flag channel must indicate a valid tracked point.
- Motion: The magnitude of the velocity vector must exceed a minimum threshold.
- Depth: The 3D position must be finite and within a configured spatial range.

Points that pass all filters are copied into a device buffer. After kernel execution, the filtered data is transferred from GPU to CPU memory. This parallel approach allows processing large images at interactive rates, even when the proportion of valid points is low.

2. DBSCAN clustering The second stage takes the filtered 6D points and performs density-based clustering using DBSCAN. This is implemented using the `mlpack` C++ library, which provides an extensible interface for clustering with custom distance functions. In this case, a metric combining spatial distance and velocity similarity is used. The motivation and details of this distance function are presented in the following subsection.

DBSCAN is configured with parameters ε and $minPts$, which are specified as ROS parameters. These control the local density required to initiate a cluster and the radius used to define neighborhoods. Clusters with fewer than the minimum size are discarded as noise. This helps remove isolated points or spurious detections that may pass the filtering stage but do not correspond to any coherent object.

3. Cluster-to-object conversion For each valid cluster found by DBSCAN, a new `WorldEntity` object is created. This class encapsulates the aggregate properties of the cluster, including its 3D bounding box, average velocity, and center position. The conversion is straightforward: all points in the cluster are passed to the constructor of the `WorldEntity`, which then computes the relevant statistics. This separation of concerns improves modularity and simplifies testing.

The high-level flow of the detection pipeline is outlined in Algorithm 4.

Algorithm 4 Object detection pipeline (`ObjectDetector::detect`)

Input: 6D motion field (OpenCV matrix)
Output: List of `WorldEntity` objects

Launch CUDA kernel to extract valid motion vectors
Transfer filtered points from GPU to CPU
if number of valid points $<$ minimum **then**
    return empty list
end if
Instantiate DBSCAN with custom distance function
Run clustering on the filtered point set
for all output clusters **do**
    if cluster size $<$ minimum allowed **then**
        discard as noise
    else
        construct `WorldEntity` from cluster
    end if
end for
return list of detected entities

A structural overview of the class is presented in Figure 3.23. The figure shows the main attributes, public interface, and key internal methods used during the detection process.

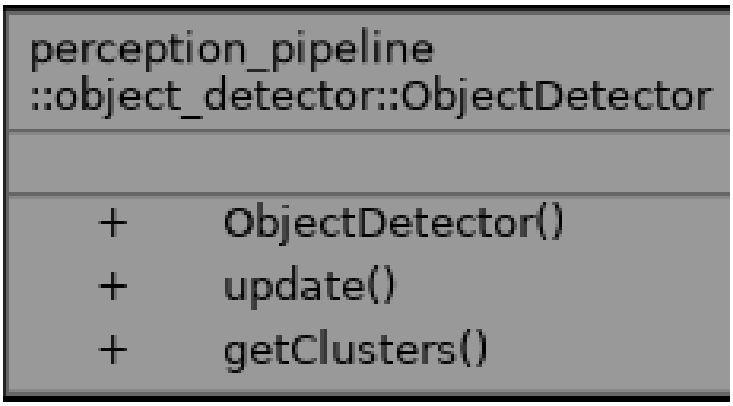


Figure 3.23: UML diagram of the `ObjectDetector` class. Source: own work

This architecture ensures that the computational workload is kept minimal on the CPU and that the more expensive filtering operations benefit from parallel execution on the GPU. The use of `mlpack` further abstracts the clustering logic, while still allowing full control over the distance metric and clustering parameters.

The following section describes the custom distance function in detail.

Custom distance function The clustering stage uses a custom distance metric that integrates both position and velocity into the similarity evaluation. This design choice addresses a core limitation of purely geometric metrics, which may erroneously group objects that are close in space but move in distinct directions. In dynamic environments, true object boundaries are often better defined by coherent motion patterns than by absolute position alone.

The distance function is implemented in the `WeightedPosVelDistance` class defined in `point_distance.hpp`, and used directly within the `mlpack::DBSCAN` routine. Each input point is assumed to be a six-dimensional vector, where the first three components represent the 3D position and the last three represent the 3D velocity. The distance between two such points, $\mathbf{a}$ and $\mathbf{b}$, is computed as:

$$d(\mathbf{a}, \mathbf{b}) = \alpha \cdot \|\mathbf{p}_a - \mathbf{p}_b\|_2 + \beta \cdot \|\mathbf{v}_a - \mathbf{v}_b\|_2 \tag{3.75}$$

where $\mathbf{p}_a, \mathbf{p}_b \in \mathbb{R}^3$ are the positions and $\mathbf{v}_a, \mathbf{v}_b \in \mathbb{R}^3$ are the velocities of the respective points. The scalar weights α and β determine the influence of each term and can be tuned at runtime via ROS2 parameters. A higher value of β increases the importance of velocity similarity in cluster formation.

The purpose of this formulation is to promote the grouping of points that not only occupy similar positions in space, but also move with similar direction and magnitude. This is critical for distinguishing between overlapping or adjacent objects that follow different trajectories. For example, two pedestrians walking side by side will likely be clustered together, while one standing still nearby will not be included unless their motion aligns.

Internally, the `Evaluate()` method partitions each input vector into position and velocity components using Armadillo's `subvec()` API. The corresponding norms are computed using `norm()` and combined with the configured weights to produce the final scalar value. The function avoids memory allocations and operates efficiently even on large datasets.

By embedding velocity coherence directly into the clustering distance, this approach ensures that object-level groupings reflect not only spatial continuity, but also temporal dynamics. This is especially important for

downstream applications such as predictive tracking, trajectory forecasting, or collision estimation, where consistent motion behavior is a strong indicator of object identity.

WorldEntity class

Each cluster produced by the DBSCAN algorithm is converted into a `WorldEntity` object, which encapsulates its geometric and kinematic properties in a compact representation. The class is initialized with a list of 3D points and corresponding velocity vectors, from which it computes the centroid, average velocity, and an oriented bounding box. These quantities are stored internally and accessed via public getter methods. The centroid is computed as the arithmetic mean of all 3D positions, and the average velocity is calculated analogously from the velocity vectors. These two features provide a representative location and motion estimate for the object.

The bounding box is computed using a Principal Component Analysis (PCA), which enables an orientation-aware description of the cluster's shape. The procedure consists of the following steps:

1. Center the input data by subtracting the mean position from each point.
2. Form a $3 \times N$ matrix containing all centered 3D points.
3. Compute the covariance matrix of the centered data.
4. Solve the eigenvalue decomposition to obtain three orthogonal basis vectors.
5. Project all points into this PCA-aligned coordinate system.
6. Determine the minimum and maximum bounds along each principal axis.
7. Reconstruct the eight corners of the bounding box in world coordinates by transforming the axis-aligned box in PCA space back to the original frame.

This results in an oriented bounding box that tightly encloses the cluster. The three axes of the box correspond to the directions of maximum variance in the data, and the box dimensions reflect the spatial spread along those axes. If the number of points is too low to perform PCA reliably, a fallback axis-aligned bounding box is computed directly from the minimum and maximum values in each coordinate dimension.

In addition to geometric information, the class assigns a unique identifier to each entity and includes utility methods for converting its contents into ROS-compatible messages. These include a method for generating a visualization marker and another for publishing the entity as part of a message array. The class diagram shown in Figure 3.24 summarizes the internal structure and public interface of the implementation.

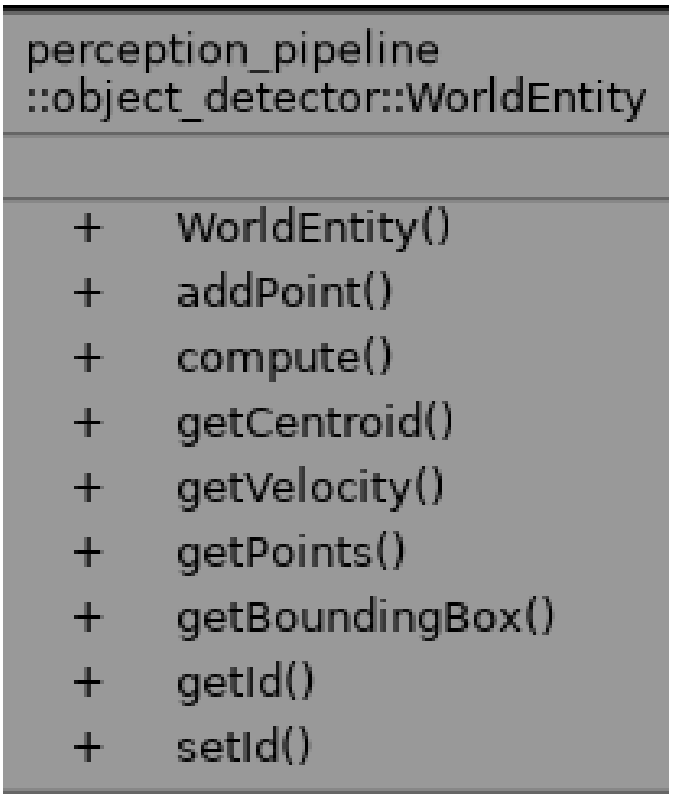


Figure 3.24: UML diagram of the `WorldEntity` class. Source: own work

This class provides a consistent abstraction between the clustering algorithm and the next stages of the perception pipeline, enabling object-level reasoning based on motion vector fields.

ROS2 integration

The `ObjectDetectorNode` serves as the ROS2 interface to the object detection module. It connects the independent detection logic with the rest of the perception pipeline and is responsible for receiving motion data, triggering clustering, and publishing object-level representations. The node subscribes to a single topic containing the 6D motion field output from the Kalman filter stage. This field is published as a `sensor_msgs::msg::Image` message with encoding `CV_32FC(7)`. Each pixel in the image stores a seven-dimensional float vector: three values for position, three for velocity, and one flag indicating if the data is valid.

Upon receiving a new message, the callback function `callback6dImage()` converts the input to a `cv::Mat`, which is then passed to the detector. The `ObjectDetector` instance processes the image and returns a list of clustered entities. These clusters are then published in two forms: as RViz visualization markers and as structured messages for downstream consumption.

The structured output is published on the `/object_clusters` topic using a custom message type `ClusteredObjectArray`. This message encapsulates all detected objects for a given frame. It contains a header, the number of clusters, and a vector of `ClusteredObject` messages. Each `ClusteredObject` includes:

- `id`: a unique integer identifier for the object
- `position`: a `geometry_msgs::Point` representing the centroid
- `velocity`: a `geometry_msgs::Vector3` with average motion

- `bb_marker`: a `visualization_msgs::Marker` for RViz
- `bounding_box`: a custom message of type `BoundingBox`

The `BoundingBox` message defines the geometry of the oriented bounding box as a list of eight 3D corners, stored as an array of `geometry_msgs::Point`. This explicit representation of the box enables further geometric reasoning in later stages of the pipeline, including volume checks, trajectory prediction, or collision assessment.

In parallel to structured outputs, the node publishes markers for visualization. These include the bounding box as a wireframe, the motion vector as an arrow, and the clustered points as spheres, all in the `camera_optical_frame`. This visual feedback is critical for debugging and validation, and is useful during live operation as well as during dataset annotation or offline replay.

All relevant parameters of the node are loaded from the ROS2 parameter server. These include the clustering radius ε, the minimum number of points `minPts`, velocity threshold, and the weights `pos_weight` and `vel_weight` for the custom distance metric. The input and output topics can also be configured via parameters to ensure compatibility with different integration setups.

The node follows standard ROS2 design conventions and uses `message_filters::Synchronizer` when required to align incoming data. The internals of the clustering remain decoupled from ROS, ensuring that the core algorithm remains portable and testable in isolation.

With this structure, the object detection node forms a robust interface layer that bridges raw motion estimation with object-level perception, providing consistent, interpretable outputs for higher-level modules such as criticallity estimation, trajectory planning, or logging and evaluation tools.

3.7 Criticallity pipeline: TTC computation

4 Experiments and Results

4.1 Camera hardware selection

4.2 Experimental setup

4.3 Perception pipeline evaluation

4.3.1 Stereo image computation

4.3.2 Optical flow computation

4.3.3 Ego motion estimation

4.3.4 Kalman filter

4.3.5 Object detection

4.4 Criticallity pipeline evaluation

5 Summary and Conclusion

- High level explanation on how the work will be structured. - What the perception and criticallity pipeline are - Go through each component.

List of Figures

List of Tables

Bibliography

[1] J. Van Brummelen, M. O'Brien, D. Gruyer and H. Najjaran, "Autonomous vehicle perception: The technology of today and tomorrow," *Transportation Research Part C: Emerging Technologies*, vol. 89, pp. 384–406, 2018, DOI: 10.1016/j.trc.2018.02.012.

[2] BMW. "*Level 3 highly automated driving available in the new BMW 7 Series from next spring.*" en. 2023. Available: https://www.press.bmwgroup.com/global/article/detail/T0438214EN/level-3-highly-automated-driving-available-in-the-new-bmw-7-series-from-next-spring?language=en [visited on 02/26/2025].

[3] Mercedes-Benz. "*Drive Pilot*," 2025. Available: https://www.mercedes-benz.de/passengercars/technology/drive-pilot.html [visited on 02/26/2025].

[4] Waymo. "*Meet the 6th-generation Waymo Driver: Optimized for costs, designed to handle more weather, and coming to riders faster than before*," 2024. Available: https://waymo.com/blog/2024/08/meet-the-6th-generation-waymo-driver [visited on 02/26/2025].

[5] Y. Li and J. Ibanez-Guzman, "Lidar for Autonomous Driving: The Principles, Challenges, and Trends for Automotive Lidar and Perception Systems," *IEEE Signal Processing Magazine*, vol. 37, no. 4, pp. 50–61, 2020, DOI: 10.1109/MSP.2020.2973615.

[6] J. Leonard, J. How, S. Teller, M. Berger, S. Campbell, et al., "A perception-driven autonomous urban vehicle," *Journal of Field Robotics*, vol. 25, no. 10, pp. 727–774, 2008, DOI: 10.1002/rob.20262.

[7] R. H. Rasshofer and K. Gresser, "Automotive Radar and Lidar Systems for Next Generation Driver Assistance Functions," in *Advances in Radio Science*, 2005, pp. 205–209, DOI: 10.5194/ars-3-205-2005.

[8] K. N. Maanyu, D. G. Raj, R. V. Krishna and D. S. B. Choubey, "A Study on Tesla Autopilot," vol. 71, no. 171, 2020.

[9] R. Szeliski, *Computer Vision: Algorithms and Applications*, (Texts in Computer Science), Cham, Springer International Publishing, 2022, ISBN: 978-3-030-34371-2 978-3-030-34372-9. DOI: 10.1007/978-3-030-34372-9.

[10] L.-H. Wen and K.-H. Jo, "Deep learning-based perception systems for autonomous driving: A comprehensive survey," *Neurocomputing*, vol. 489, pp. 255–270, 2022, DOI: 10.1016/j.neucom.2021.08.155.

[11] A. Alfarano, L. Maiano, L. Papa and I. Amerini, "Estimating optical flow: A comprehensive review of the state of the art," *Computer Vision and Image Understanding*, vol. 249, p. 104160, 2024, DOI: 10.1016/j.cviu.2024.104160.

[12] S. Lucey, R. Navarathna, A. B. Ashraf and S. Sridharan, "Fourier Lucas-Kanade Algorithm," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 6, pp. 1383–1396, 2013, DOI: 10.1109/TPAMI.2012.220.

[13] U. Franke, C. Rabe, H. Badino and S. Gehrig, "6D-Vision: Fusion of Stereo and Motion for Robust Environment Perception," in *Pattern Recognition*, 2005, pp. 216–223, ISBN: 978-3-540-31942-9. DOI: 10.1007/11550518_27.

[14] P. Karle, T. Betz, M. Bosk, F. Fent, N. Gehrke, et al. “*EDGAR: An Autonomous Driving Research Platform – From Feature Development to Real-World Application*," arXiv:2309.15492 [cs]. Jan. 2024. DOI: 10.48550/arXiv.2309.15492.

[15] S. Bauer, “Objektdetektion und Bewegungsprädiktion mittels eines Stereokamerasystems beim teleoperierten Fahren," MA thesis, TUM, Munich, 2014.

[16] A. Rotar, “Weiterentwicklung und echtzeitfähige Umsetzung eines Stereo-Vision-basierten Bewegungsprädiktionssystems für die aktive Sicherheit von teleoperierten Fahrzeugen," MA thesis, TUM, Munich, 2016.

[17] C. Pastor, “Validation of an Obstacle Detection Approach for Teleoperation of Automated Vehicles," BSc. Thesis, TUM, 2021.

[18] L. Westhofen, C. Neurohr, T. Koopmann, M. Butz, B. Schütt, et al., “Criticality Metrics for Automated Driving: A Review and Suitability Analysis of the State of the Art," *Archives of Computational Methods in Engineering*, vol. 30, no. 1, pp. 1–35, 2023, DOI: 10.1007/s11831-022-09788-7.

[19] S. Heidari, M. Rogers and P. J. Delmas, “(PDF) An Improved Quantum Solution for the Stereo Matching Problem," in *ResearchGate*, DOI: 10.1109/IVCNZ54163.2021.9653310. Available: https://www.researchgate.net/publication/357424395_An_Improved_Quantum_Solution_for_the_Stereo_Matching_Problem [visited on 04/26/2025].

[20] J. Karathanasis, D. Kalivas and J. Vlontzos, “Disparity estimation using block matching and dynamic programming," in *Proceedings of Third International Conference on Electronics, Circuits, and Systems*, 1996, 728–731 vol.2, DOI: 10.1109/ICECS.1996.584465. Available: https://ieeexplore.ieee.org/abstract/document/584465 [visited on 04/26/2025].

[21] H. Hirschmuller, “Stereo Processing by Semiglobal Matching and Mutual Information," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, DOI: 10.1109/TPAMI.2007.1166.

[22] R. A. Hamzah and H. Ibrahim, “Literature Survey on Stereo Vision Disparity Map Algorithms," *Journal of Sensors*, vol. 2016, no. 1, p. 8742920, 2016, DOI: 10.1155/2016/8742920. Available: https://onlinelibrary.wiley.com/doi/abs/10.1155/2016/8742920 [visited on 04/26/2025].

[23] D. C. Niehorster, “Optic Flow: A History," *i-Perception*, vol. 12, no. 6, p. 20416695211055766, 2021, DOI: 10.1177/20416695211055766. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8652193/.

[24] G. Boesch. “*Optical Flow - Everything You Need to Know in 2025*," en-US. Oct. 2024. Available: https://viso.ai/deep-learning/optical-flow/ [visited on 11/12/2024].

[25] D. Fortun, P. Bouthemy and C. Kervrann, “Optical flow modeling and computation: A survey," *Computer Vision and Image Understanding*, vol. 134, pp. 1–21, 2015, DOI: 10.1016/j.cviu.2015.02.008. Available: https://linkinghub.elsevier.com/retrieve/pii/S1077314215000429 [visited on 11/13/2024].

[26] O. Zvorișteanu, S. Caraiman and V. Manta, “Speeding Up Semantic Instance Segmentation by Using Motion Information," *Mathematics*, vol. 10, p. 2365, 2022, DOI: 10.3390/math10142365.

[27] P. Fischer, A. Dosovitskiy, E. Ilg, P. Häusser, C. Hazirbas, et al., “FlowNet: Learning Optical Flow with Convolutional Networks," *CoRR*, vol. abs/1504.06852, 2015. arXiv: 1504.06852. Available: http://arxiv.org/abs/1504.06852.

[28] J. L. Barron, D. J. Fleet and S. S. Beauchemin, “Performance of optical flow techniques," *International Journal of Computer Vision*, vol. 12, no. 1, pp. 43–77, 1994, DOI: 10.1007/BF01420984. Available: https://doi.org/10.1007/BF01420984.

[29] B. K. P. Horn and B. G. Schunck, “Determining optical flow," *Artificial Intelligence*, vol. 17, no. 1, pp. 185–203, 1981, DOI: 10.1016/0004-3702(81)90024-2.

[30] J. Sánchez, E. Llopis and G. Facciolo, “TV-L1 optical flow estimation," *Image Processing On Line*, vol. 3, pp. 137–150, 2013, DOI: 10.5201/ipol.2013.26.

[31] G. Farnebäck, “Two-Frame Motion Estimation Based on Polynomial Expansion," 2003, pp. 363–370, ISBN: 978-3-540-40601-3. DOI: 10.1007/3-540-45103-X_50.

[32] R. Szeliski, “Computer Vision: Algorithms and Applications," 2021.

[33] H. Moravec, “Obstacle Avoidance and Navigation in the Real World by a Seeing Robot Rover," Pittsburgh, PA, 1980.

[34] D. Nister, “An efficient solution to the five-point relative pose problem," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 26, no. 6, pp. 756–770, 2004, DOI: 10.1109/TPAMI.2004.17.

[35] K. L. Lim and T. Bräunl. “*A Review of Visual Odometry Methods and Its Applications for Autonomous Driving*," arXiv:2009.09193 [cs]. Sept. 2020. DOI: 10.48550/arXiv.2009.09193.

[36] E. Marchand, H. Uchiyama and F. Spindler, “Pose Estimation for Augmented Reality: A Hands-On Survey | Request PDF," *ResearchGate*, 2024, DOI: 10.1109/TVCG.2015.2513408. Available: https://www.researchgate.net/publication/287348989_Pose_Estimation_for_Augmented_Reality_A_Hands-On_Survey [visited on 04/29/2025].

[37] OpenCV. “*OpenCV: ORB (Oriented FAST and Rotated BRIEF)*," Available: https://docs.opencv.org/4.x/d1/d89/tutorial_py_orb.html [visited on 04/27/2025].

[38] OpenCV. “*OpenCV: FAST Algorithm for Corner Detection*," Available: https://docs.opencv.org/3.4/df/d0c/tutorial_py_fast.html [visited on 04/27/2025].

[39] OpenCV. “*OpenCV: BRIEF (Binary Robust Independent Elementary Features)*," Available: https://docs.opencv.org/3.4/dc/d7d/tutorial_py_brief.html [visited on 04/27/2025].

[40] Deep. “*Introduction to FAST (Features from Accelerated Segment Test) — deepanshut041*," https://medium.com/@deepanshut041/introduction-to-fast-features-from-accelerated-segment-test-4ed33dde6d65. [Accessed 29-04-2025].

[41] OpenCV. “*OpenCV: Perspective-n-Point (PnP) pose computation*," 2024. Available: https://docs.opencv.org/4.x/d5/d1f/calib3d_solvePnP.html [visited on 04/29/2025].

[42] OpenMP. “*Reference Guides - OpenMP — openmp.org*," https://www.openmp.org/resources/refguides/. [Accessed 02-05-2025]. 2025.

[43] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems," *Transactions of the ASME–Journal of Basic Engineering*, vol. 82, no. Series D, pp. 35–45, 1960.

[44] M. Ribeiro and I. Ribeiro. “*Kalman and Extended Kalman Filters: Concept, Derivation and Properties*," Apr. 2004.

[45] C. Rabe, “Detection of Moving Objects by Spatio-Temporal Motion Analysis," 2011.

[46] H. Cheng and K. C. Gupta, “An Historical Note on Finite Rotations," *Journal of Applied Mechanics*, vol. 56, no. 1, pp. 139–145, 1989, DOI: 10.1115/1.3176034. Available: https://doi.org/10.1115/1.3176034.

[47] J. S. Dai, “Euler–Rodrigues formula variations, quaternion conjugation and intrinsic connections," *Mechanism and Machine Theory*, vol. 92, pp. 144–152, 2015, DOI: 10.1016/j.mechmachtheory.2015.03.004. Available: https://www.sciencedirect.com/science/article/pii/S0094114X15000415 [visited on 05/02/2025].

[48] M. Ester, H.-P. Kriegel, J. Sander and X. Xu, “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," in *Knowledge Discovery and Data Mining*, 1996. Available: https://api.semanticscholar.org/CorpusID:355163.

[49] A. Mehle, B. Likar and D. Tomaževič, “In-line recognition of agglomerated pharmaceutical pellets with density-based clustering and convolutional neural network," *IPSJ Transactions on Computer Vision and Applications*, vol. 9, 2017, DOI: 10.1186/s41074-017-0019-2.

Appendix

A Chapter Anhang

A.1 Section Anhang

A.1.1 Subsection Anhang

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

B Chapter Anhang 2

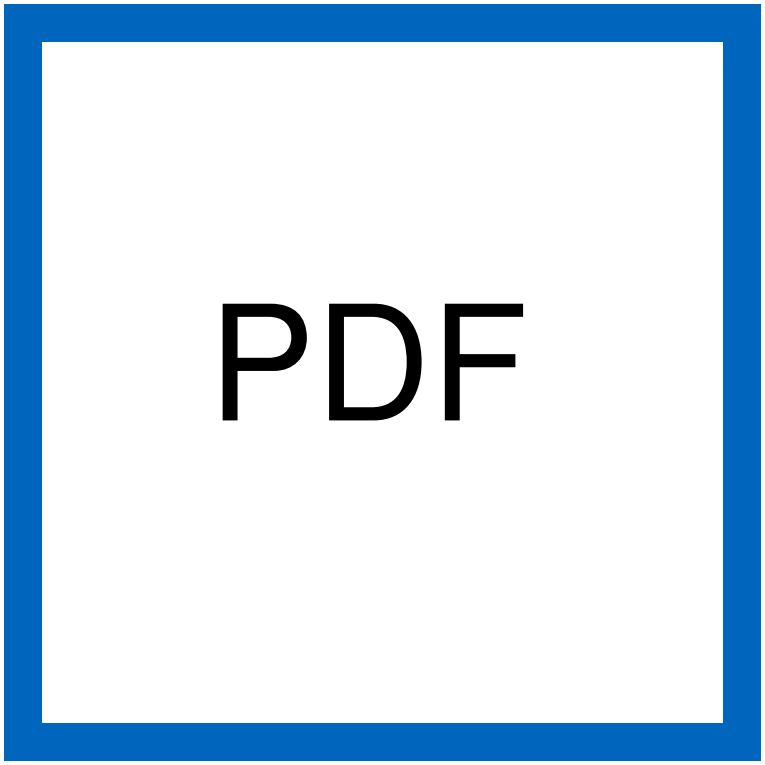


Figure B.1: Bild im Anhang

Table B.1: Tabelle im Anhang

Spalte 1	Spalte 2
...	...